



Piszcie poprawnie - polskie znaki i ortografia to nie są mityczne potwory!



LOL

1. Omów mechanizm journalingu. Wymień nazwy systemów plików, w których jest stosowany.
2. Omów proces uruchamiania komputera z BIOSem (od wciśnięcia przycisku zasilania do załadowania systemu operacyjnego)
3. Przedstaw charakterystykę systemów opartych o EFI
4. Wyjaśnij różnicę między liniowym a fizycznym adresem w pamięci

operacyjnej

5. Wymień metody ukrywania danych (plików) w systemach plików i opisz pokrótce ideę każdej z nich:

6. Podaj cele stosowania struktury GDT

7. Opisz rolę bloku PEB (blok wykonawczy procesu)

8. Opisz przepływ sterowania podczas tworzenia procesu w systemie Windows XP

9. Jaka rolę w systemie plików NTFS pełni nadrzędna tablica plików (MFT)

10. Budowa i rola rekordu plikowego w NTFS.

11. W jaki sposób tablica alokacji plików (FAT) opisuje położenie plików na dysku?

12. Omówić strukturę partycji rozszerzonej na dysku.

13. Opisz mechanizm obsługi przerwania na poziomie jądra w systemie Windows XP. Wyjaśnij rolę zmiennej IRQL.

14. Przedstaw działanie i zastosowanie odwzorowywania plików (file mapping). Podaj przykładowe wykorzystanie tego mechanizmu w praktyce.

15. Opisz proces ładowania sterownika WDM do pamięci RAM podczas rozruchu systemu.

16. Przedstaw klasyfikację sterowników w systemie Windows XP i podaj ich charakterystyki.

17. Sposoby rejestracji sterownika w systemie Windows.

18. Omówić implementację pamięci wirtualnej za pomocą segmentacji.

19. Jakie zalety ma realizacja pamięci wirtualnej za pomocą stronicowania w porównaniu z realizacją za pomocą segmentacji?

20. W jaki sposób w architekturze IA-32 została zrealizowana koncepcja rejestru bazowego i granicznego jako metody izolacji procesów?

21. Opisz mechanizm wywołania funkcji systemowej na poziomie procesora (rejestru GDT)

22. Opisz mechanizm wywołania funkcji systemowej na poziomie jądra systemu operacyjnego Windows XP

23. Przedstaw działanie procesu planisty w systemie Windows XP

24. Wymień i scharakteryzuj metody łączenia kodu bibliotecznego z kodem programu

25. Opisz budowę pliku PE

26. Przedstaw kolejne kroki w komunikacji pomiędzy aplikacją użytkową a sprzętem w systemie Windows

27. Opisz proces ładowania programów do pamięci operacyjnej

28. Omów budowę podstawowego szkieletu sterownika urządzeń WDM w systemie Windows.

29. Ile stosów ma procedura w trybie chronionym? Wyjaśnij dlaczego tak się dzieje.

30. Opisz proces ładowania sterowników dla urządzeń PnP.

31. Wymień i opisz sposoby komunikacji aplikacji użytkowej ze sterownikiem WDM.
32. Omów typowe działanie systemu operacyjnego przy błędach strony (page fault handling).
33. Na czym polega zagadnienie DLL-hell. Wymień sposoby pozwalające na uniknięcie tego problemu.
34. Wymień trzy podstawowe struktury systemowe wykorzystywane przy budowie i działaniu sterowników urządzeń i podaj ich znaczenie
35. Podaj różnicę pomiędzy zadaniami (jobs), procesami, wątkami i włóknami.
36. Jaka jest rola mechanizmu DPC i na czym on polega ?
37. Jaka jest rola mechanizmu APC i na czym on polega ?
38. Wyjaśnij następujące pojęcia MBR, VBR, DDK, IDT, GDTR.
39. Wymień 5 różnych systemów plików.
40. Omów mechanizm przełączania stosu używany w trakcie przyjmowania przerwania.
41. Przedstaw klasyfikację maszyn wirtualnych i opisać ich charakterystyki
42. Opisz strukturę bio zdefiniowaną w <linux/bio.h>
43. Porównaj poznane schedulery dla urządzeń blokowych I/O w systemie Linux.
44. Czym charakteryzuje się problem „write starving reads” i gdzie występuje ?
45. Przedstaw strukturę i-node dla systemu plików ext3.
46. Podaj blok oraz offset, gdzie znajduje się bajt 6 000 000 w systemie plików ext3. Przedstaw tok obliczania.
47. Przedstaw proces zamykania procesu w systemie Linux (do_exit()).
48. Co to jest proces zombie i jak należy z nim postępować?
49. W jaki sposób w systemie Linux tworzony jest nowy proces?
50. Opisz CFS.
51. Zaprezentuj schematycznie i opisz budowę kernela w systemie Linux.
52. Podaj przewagę sterowników kernel-space nad sterownikami user-space w systemie Linux.
53. Slab allocator i jego możliwe zachowania.
54. Przedstaw szczegółowo oraz porównaj metody alokacji pamięci na poziomie kernela w systemie Linux.
55. Przedstaw działanie wybranych flag GFP.
56. Co to jest etyka? :)

Pytania z kolokwium:

- 1) Opisz mechanizm wywołania f.systemowej na poziomie jądra XP
- 2) Jaka jest rola mechanizmu DPC i na czym on polega

- 3) Jaka role w NTFS pełni MFT
- 4) Przedstaw proces zamykania procesu w Linux (`do_exit()`)
- 5) Opisz CFS
- 6) Opisz budowę pliku PE
- 7) Dlaczego w trybie chronionym x86 zostały wyróżnione call gates a nie zostały wykorzystane deskryptory przerwań.

1. Omów mechanizm journalingu. Wymień nazwy systemów plików, w których jest stosowany.

Dziennik transakcji (kronikowanie – ang. *journaling*) jest prowadzony w celu zwiększenia bezpieczeństwa danych dyskowych; gdy dowolny plik jest tworzony, modyfikowany lub usuwany, system NTFS dodaje odpowiedni rekord do dziennika transakcji (dziennika zmian); każdy rekord w dzienniku zmian zajmuje w przybliżeniu 80-100 bajtów; faktyczny zapis danych następuje po rejestracji w dzienniku; w przypadku niepowodzenia w zapisie (wskutek błędów transmisji lub uszkodzeń) transakcje są wycofywane z dysku;

Miedzy innymi omawiany dziennik może być przydatny do tworzenia przyrostowych kopii zapasowych; używa się do tego celu odpowiednich funkcji API, które są udostępniane przez system.

Usunięcie pliku w UNIX:

-usunięcie wpisu w katalogu

-oznaczenie iwęzła i pliku jako wolne/usunięte

Obie te operacje muszą być wykonane jako operacja atomowa.

Systemy w których jest używany: BeFS, ext3, ext4, GFS, HFS+, JFS, NTFS, ReiserFS, Reiser4, SFS, XFS, UFS.

2. Omów proces uruchamiania komputera z BIOSem (od wciśnięcia przycisku zasilania do załadowania systemu operacyjnego)

- BIOS POST – pierwsza wykonana czynność. Zazwyczaj testuje integralność sprzętu (rejstry, pamięć, klawiatura etc.) i wyłącza niemaskowalne przerwanie.
- Załadowanie MBR – Master Boot Record – pierwszy sektor dysku twardego zawierający program rozruchowy
- Załadowanie VBR – Volume Boot Record – jak wyżej, tyle, że na partycjach
- odpalenie bootmgr – umożliwia załadowanie OSa (wyświetla też menu wyboru)
- Przetworzenie BCD – Boot Configuration Data – dane do użycia w trakcie bootowania
- winload.exe – bootloader windowsa przejmuje pałeczkę, on ładuje wszystko aż do sterowników boot-class
- ładowanie SYSTEM hive – zawiera ustawienia nie przyporządkowane do żadnego użytkownika
- samosprawdzenie integralności winload.exe
- załadowanie i sprawdzenie ntoskrnl.exe, hal.dll – jądro systemu i Hardware Abstraction Layer
- ładowanie, weryfikacja i inicjalizacja importów ntoskrnl.exe
- skanowanie rejestrów
- ładowanie, weryfikacja i inicjalizacja sterowników boot-class
- ntoskrnl.exe jest inicjalizowany – w końcu. Przejmuje pałeczkę.
- inicjalizacja podsystemów, zbudowanie struktury systemu
- skanowanie rejestrów
- ładowanie, weryfikacja i inicjalizacja sterowników i usług „system class”
- smss.exe – Session Manager Subsystem – tworzy zmienne środowiskowe i generalnie przejmuje

pałeczkę od tego momentu

- odpalenie aplikacji startupowych określonych przez wpis BootExecute w rejestrze
- załadowanie jądra podsystemu Win32K.sys
- załadowanie znanych DLL'ek
- podział:
- Session 0 smss.exe odpala
 - csrss.exe – client-server run-time subsystem – odpowiada za wątki m.in
 - wininit.exe – program inicjalizujący windowsa
 - lsass.exe, lsm.exe, services.exe -jakieś tam rzeczy dla windowsa
- Session 1 smss.exe odpala
 - csrss.exe - client-server run-time subsystem – odpowiada za wątki m.in
 - winlogon.exe - wiadomo
 - logonUI.exe – jeszcze bardziej wiadomo
 - userinit.exe – program inicjalizujący usera

3. Przedstaw charakterystykę systemów opartych o EFI

EFI: Extensible Firmware Interface - definiuje interfejs pomiędzy systemem operacyjnym a firmwarem sprzętu, następcą BIOS.

Systemy EFI:

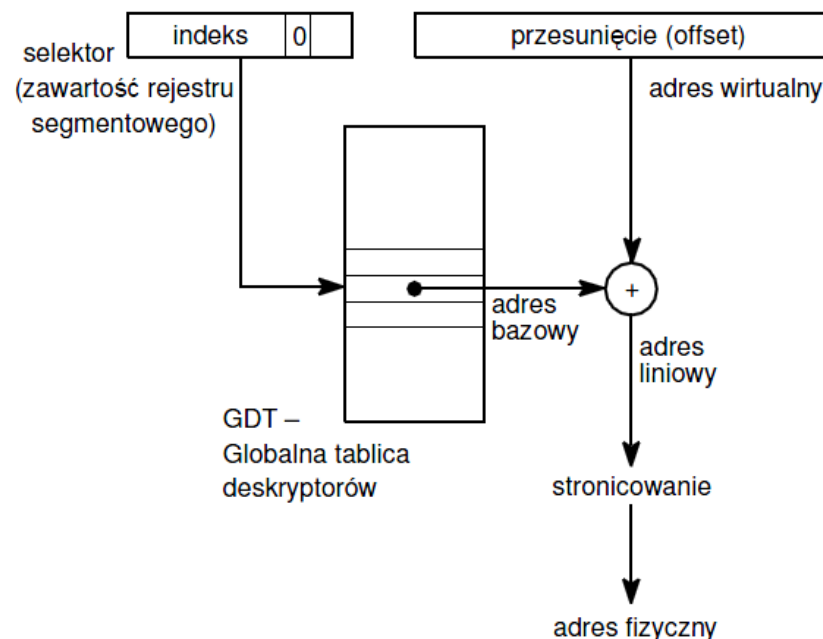
- są niezależne od implementacji sprzętu
- nie wymagają trybu rzeczywistego procesora do rozruchu
- wykorzystują płaski model pamięci, gdzie cała dostępna pamięć może być zaadresowana
- nie nakładają limitów na całkowity rozmiar pamięci ROM. Sterowniki i aplikacje mogą być ładowane z dowolnego źródła w dowolne miejsce przestrzeni adresowej.
- Zawierają dodatkową powłokę (EFI shell)
- Ewoluuja od VGA do UGA (Universal Graphics Adapter)
- Jako pierwsza załadowana zostaje cienka warstwa PEI, która wykonuje większość rzeczy związanych z rozruchem komputera (inicjalizacja chipsetu, pamięci, enumeracja szyn danych etc.)
- Następnie EFI przygotowuje DXE (Driver Execution Environment) by zapewnić podstawowe funkcje, których mogą użyć sterowniki EFI
- Wykrywane są partycje
- Następuje uruchomienie bootloadera i jak wszystko ok, to przekazujemy sterowanie do OSa. W przeciwnym wypadku uruchamiany jest EFI Shell.
- Umożliwiają bootowanie z dużych dysków (>2TB)
- Posiada własny boot loader
- Szybszy proces bootowania
- Wsparcie dla sieci, bardziej przyjazny interfejs graficzny.

4. Wyjaśnij różnicę między liniowym a fizycznym adresem w pamięci operacyjnej

Adres fizyczny jest adresem bezpośrednio określającym położenie segmentu w pamięci, przez ten adres maszyna musi się do odpowiedniego segmentu odwoływać.

Adres liniowy natomiast składa się z numeru katalogu(10-bit), odnoszącego się do Directory Entry w Page Directory, numeru tabeli(10-bit), odnoszącego się do Table Entry w Page Table i offsetu(12-bit). Taki adres, zapisany w Dekryptorze Segmentu w Global Descriptor Table może zostać wykorzystany przez sprzęt dopiero po przetłumaczeniu z użyciem wyżej wymienionych stron na adres fizyczny.

W przypadku wyłączenia stronicowania, adres liniowy jest tożsamy adresowi fizycznemu wyznaczonemu w opisany sposób.



5. Wymień metody ukrywania danych (plików) w systemach plików i opisz pokrótce ideę każdej z nich:

Out of band: zapis w niewykorzystanym i niewidocznym przez system miejsce. Wynika z rozmiaru klastra - pojemność dysku jest skokowa i przy partycjonowaniu może nam zostać trochę nieużywanego miejsca.

In-band: zapis w miejscu zarezerwowanym dla pliku - np klaster 4kb, plik 1kb i zostaje wolne 3kb (plik nawet jednobitowy zajmuje na dysku cały klaster)

Ukrywanie partycji (np w laptopach partycja do odzyskiwania systemu) - typ partycji ustawiamy na jakiś swój i wtedy system nie wie co to jest i zlewa.d

Ukrywanie w warstwie aplikacyjnej: pewnie ukrywamy jakieś dane w obszarze, gdzie niby aplikacja ma być- może chodzi o coś w stylu zahardkodowania danych?

6. Podaj cele stosowania struktury GDT

- uzyskanie liniowej przestrzeni pamięci, niezależnie od fizycznego rozproszenia danych

- umożliwienie obsługi większych ilości pamięci niż przy adresowaniu fizycznym
- zwiększenie funkcjonalności i kontroli nad tym, co się dzieje - dodanie opisów segmentów – kod/dane, opisy przywilejów, czy segment jest w pamięci
- zwiększenie bezpieczeństwa

7. Opisz rolę bloku PEB (blok wykonawczy procesu)

Process Environment Block zawiera struktury danych (sam w sumie też nią jest), które mają zastosowanie dla obsługi całych procesów, takie jak kontekst globalny, parametry startowe, struktury danych dla loadera obrazu programu, adres bazowy obrazu programu, obiekty synchronizujące, informacje o tym, czy program jest aktualnie debugowany, oraz wiele innych. Duża część tej struktury danych nie jest udokumentowana przez Microsoft.

8. Opisz przepływ sterowania podczas tworzenia procesu w systemie Windows XP

- Otwarcie pliku.exe i utworzenie obiektów sekcji
- utworzenie obiektów procesu
 - Ustawienie bloku EPROCESS
 - Utworzenie inicjalnej przestrzeni adresowej
 - Utworzenie bloku kernela procesu
 - Ukończenie tworzenia przestrzeni adresowej
 - Ustawienie PEB
 - Ukończenie konfiguracji EPROCESS
- Utworzenie obiektu wątku
- Powiadomienie podsystemu (csrss) o nowym procesie
- Utworzenie wątku głównego
- Utworzenie kontekstu i uruchomienie wątku

Proces to kontener z zasobami używanymi podczas wykonywania instancji programu.

Proces = exe + ProcessControlBlock (id procesu + wartości rejestrów + obszar pamięci dla procesu + otwarte pliki/IO) + stos

9. Jaką rolę w systemie plików NTFS pełni nadrzędna tablica plików (MFT)

Master File Table (MFT) -metaplik zawiera wpisy (rekordy plikowe) opisujące każdy plik/katalog znajdujący się na partycji NTFS w postaci *rekordu plikowego* (1024 -4096 bajtów); rekord plikowy składa się z nagłówka oraz sekwencji występujących po sobie atrybutów w postaci pary (atrybut, wartość);

Charakterystycznym elementem *nagłówka rekordu plikowego* są znaki „FILE” (w kodzie ASCII) umieszczone na początku nagłówka;

Nagłówek zawiera kilkanaście pól o różnym przeznaczeniu, m.in. podany jest faktyczny rozmiar rekordu plikowego (4-bajtowe pole o offsecie 18H) oraz zaalokowany rozmiar rekordu plikowego (4-bajtowe pole o offsecie 1CH) — w omawianym dalej przykładzie rekordu plikowego pola te zawierają liczby 168H i 400H; tak więc przykładowy rekord plikowy zajmuje 1024 bajty

10. Budowa i rola rekordu plikowego w NTFS.

Atrybuty w rekordzie plikowym:

- Informacje standardowe – znaczniki czasu, ilość łączy, prawa dostępu
- Nazwa pliku/katalogu
- Informacje o woluminie – nazwa woluminu
- Dane – dane pliku
- Indeks główny – używany do implementowania katalogów i indeksów
- Mapa bitowa – dane tego atrybutu zawierają pole bitowe

Rodzaje atrybutów:

- rezydentne - ich rozmiar nie przekracza rozmiaru rekordu plikowego

-nierezydentne - większe od rozmiaru rekordu plikowego, wówczas w miejscu danych znajduje się lista wskaźników do kolejnych części danych.

11. W jaki sposób tablica alokacji plików (FAT) opisuje położenie plików na dysku?

File allocation table zawiera informacje o stanie wszystkich klasterów obszaru danych dysku. Każda pozycja tablicy używana przez plik wskazuje następną pozycję w FAT dla tego pliku (metoda listowa). Sekwencja numerów kolejnych pozycji jest taka sama jak numery wiązek składających się na cały plik. Pierwsze dwie pozycje w tablicy są zarezerwowane. Trzecia zawiera informację o pierwszym klasterze obszaru danych (numer 2). Każdy wpis ma 32bajty i zawiera m.in:

- krótką nazwę pliku
- czas i datę modyfikacji/utworzenia/dostępu
- atrybuty pliku (ro,ukryty,systemowy,etykieta dysku, katalog, to długa nazwa)
- rozmiar pliku

12. Omówić strukturę partycji rozszerzonej na dysku.

Pojedyncza tablica partycji może pomieścić informacje tylko o 4 dyskach logicznych. Celem zwiększenia tej liczby zdefiniowano partycję rozszerzoną w skład, której może wchodzić wiele dysków logicznych, które przez użytkownika są widziane, jako zwykłe dyski. Partycje rozszerzone są stosowane w systemie Windows.

Na początku partycji rozszerzonej znajduje się sektor o specyficznej strukturze, w którym zawarta jest tablica partycji, położona w tym samym miejscu, co w przypadku MBR'a. W tablicy tej wykorzystywane są tylko dwie pierwsze pozycje.

Pierwszy element tablicy partycji wskazuje adres początkowy dysku logicznego.

Drugi – sektor dyskowy o takiej samej strukturze, stanowiący kolejny element listy dysków logicznych.

26 619 768	Dysk logiczny F: (2 GB)			
		0	0	4
		0	0	3
		0	0	2
	07H	63	4 096 512	1
26 619 705				
18 426 618	Dysk logiczny E: (4 GB)			
		0	0	4
		0	0	3
	05H	10 233 405	4 096 575	2
	07H	63	8 193 087	1
18 426 555				
16 386 363	Dysk logiczny D: (1 GB)			
		0	0	4
		0	0	3
	05H	2 040 255	8 193 150	2
	0BH	63	2 040 192	1
16 386 300				

x

Element zawierający 0 w polu identyfikatora systemu operacyjnego druga kolumna wskazuje koniec tej listy.

13. Opisz mechanizm obsługi przerwania na poziomie jądra w systemie Windows XP. Wyjaśnij rolę zmiennej IRQL.

Wszystkie przerwanie obsługiwane są przez systemowy dispatcher, który ma swoją IDT (interrupt dispatch table). IDT dispatchera jest systemowym odpowiednikiem IDT (interrupt descriptor table) zwykłego, procesorowego (to taka abstrakcja nad sprzętem).

- Na podstawie IDT (interrupt dispatch table) uruchamia odpowiednie ISR (interrupt service routine), która obsługuje przerwanie
 - IRQL - poziom, priorytet przerwania (*Interrupt Request Level ???-tak*)
 - Tablice IDT pokazuje jakie ISR odpowiadają za dany IRQL
- Kernel dostarcza ISR dla wielu systemowych przerw (zegarowe, związane z zasilaniem)
- Inne ISR dostarczane są przez sterowniki (zazwyczaj urządzeń)
 - Sterownik wiąże ISR z IRQL po przez utworzenie obiektu interrupt object i wysyłając go do kernela
 - Jądro łączy ten obiekt z odpowiednim wierszem tablicy IDT i gdy przerwanie wystąpi - uruchamia odpowiednią ISR dzięki temu obiektowi
 - Więcej niż jeden obiekt może być dołączony do danego poziomu IRQL w tablicy
- Przerwania z wyższym IRQL mają pierwszeństwo
- Można odroczyć wykonanie procedury za pomocą innego przerwania - DPC - deferredProcedure Calls
 - Wykorzystuje się to do wykonania kodu przy przejściu z wyższego poziomu IRQL na niższy

Opis z techneta:

Przerwania w systemie Windows są relizowane poprzez wirtualizację obsługi przerwania (przerwania od sprzętu są dostarczane w odpowiedniej formie z warstwy HAL).

Przerwania mogą mieć różny priorytet (określa to zmienna IRQ - Interrupt Request Level), przerwania o niższym priorytecie mogą zostać przerwane (zamaskowane) poprzez przerwanie o wyższej wartości IRQ.

Kiedy wystąpi przerwanie, jest ono obsługiwane przez funkcję ISR (Interrupt Service Routine).

Struktura IDT (Interrupt Dispatch Table - związana z konkretnym procesorem) kontroluje która funkcja obsługuje przerwanie z danym IRQ.

Sterowniki łączą ISR z konkretnym IRQ poprzez stworzenie obiektu przerwania (interrupt object) i przekazując go do kernela. Łączy go on z odpowiednim wpisem w tablicy IDT (z każdym IRQ może związane być więcej niż 1 przerwanie), w przypadku odpięcia sterownika(urządzenia) obiekt przerwania jest poprostu usuwany z tablicy.

<http://technet.microsoft.com/en-us/library/cc767887.aspx>

<http://mbs3.org/wdmpproject/intspri.html>

14. Przedstaw działanie i zastosowanie odwzorowywania plików (file mapping). Podaj przykładowe wykorzystanie tego mechanizmu w praktyce.

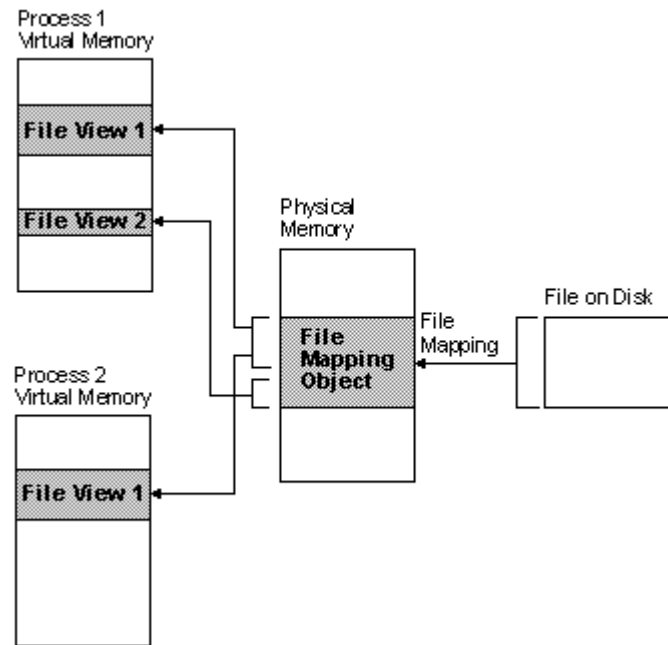
//wydaje mi się, że tutaj na początku plik mapowany jest do pamięci operacyjnej, a następnie widok
//jest mapowany do wirtualnej przestrzeni adresowej procesu. Tak przynajmniej wynika z tego /
/rysunku :P Nawet to by było logiczne, bo jeżeli dany plik przypiszemy do przestrzeni adresowej /
/jednego procesu, to przecież inny proces nie może odwołać się do przestrzeni innego (access //
violation :P)

Mapowanie pliku przypisuje zawartość pliku do części wirtualnej przestrzeni adresowej procesu.

Mapowanie pozwala procesowi korzystać z dostępu swobodnego oraz sekwencyjnego do pliku.

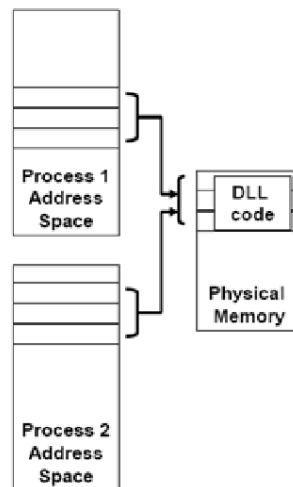
Tworzenie *widoku* (fragment wirtualnej przestrzeni adresowej) pozwala na efektywną pracę z dużymi plikami, np. baz danych – nie ma potrzeby mapowania całego pliku, a tylko jego fragmentu. Wątki mogą używać pamięci zmapowanego pliku do współdzielenia danych.

Używanie mapowania poprawia wydajność, bo korzystamy z widoku, który znajduje się w pamięci, a nie bezpośrednio z pliku na dysku. Ponadto nawet jeśli rozmiar pliku jest większy od rozmiaru pamięci możemy swobodnie się do niego odwoływać.



Relationship between the file on disk, a file mapping object, and a file view.

Mapowanie kodu bibliotek



Zastosowanie: mapowanie kodu bibliotek DLL.

15. Opisz proces ładowania sterownika WDM do pamięci RAM podczas rozruchu systemu.



Nie wiem o co chodzi dlatego wklejam

zdjęcie kota z serem na głowie

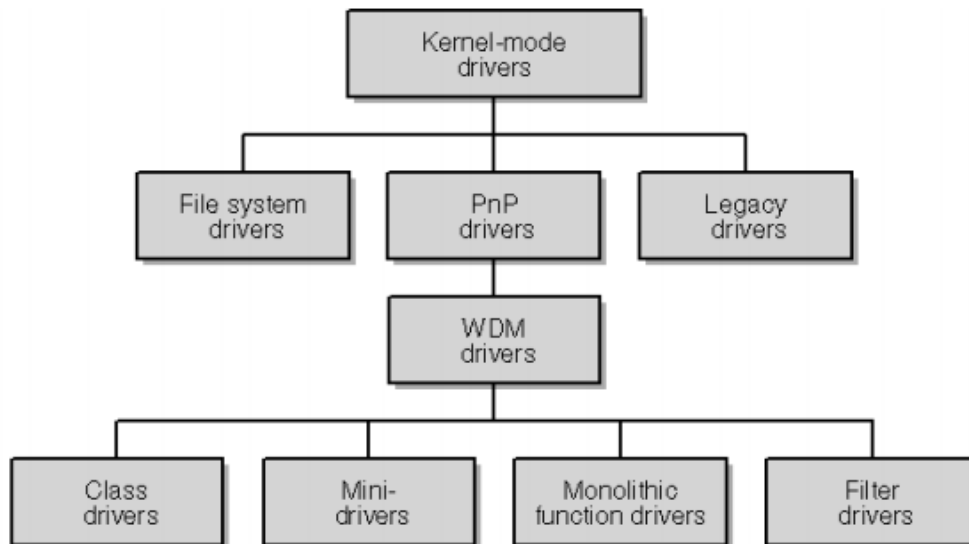
www.demotywatory.pl

Wzięte z Yeti :P (u nas to jest pytanie 30)

- Nie mam pojęcia czym to się różni od ładowania sterownika PnP z pytania 36.
- Różni się to pierwszą częścią ładowania sterownika, związaną z odpytywaniem sprzętu przez magistralę by załadować sterownik.

16. Przedstaw klasyfikację sterowników w systemie Windows XP i podaj ich charakterystyki.

Klasyfikacja sterowników dla Windows XP



- Sterowniki systemu plików: implementują system plików dla dysku lokalnego lub podłączonego sieciowo.
- Sterowniki typu Legacy: Kontrolują sprzęt bez żadnego wsparcia od strony systemu operacyjnego (Sterowniki dla nieistniejącego sprzętu są dodane do systemu właśnie jako legacy drivers)
- Sterowniki PnP: Implementują protokół Plug and Play.
- Sterowniki WDM (Windows Driver Model) to sterowniki PnP, które dodatkowo implementują protokoły zarządzania energią i są zgodne zarówno z Windows 98/ME i Windows 2K/XP. (W praktyce nie zawsze oferują systemowi wszystkich funkcji).
- Sterowniki klas: Łączą wspólną funkcjonalność dobrze zdefiniowanych klas urządzeń (np. kart sieciowych NDIS class drivers). Producenci sprzętu w większości nie piszą własnych sterowników klas, ale używają jednego z dostarczonych przez Microsoft.
- Minidriver: Pisane przez producentów jako wsparcie dla sterowników klas.
- Sterowniki monolityczne (Monolithic function driver): jeden sterownik dla całego urządzenia (systemy typu embedded).
- Sterowniki Filtrów (filter drivers): Dodają i/lub modyfikują zachowanie urządzenia

17. Sposoby rejestracji sterownika w systemie Windows.

- ZwSetSystemInformation() -Pozyskujemy adres funkcji z ntdll.dll za pomocą GetProcAddress. Nie można usunąć sterownika aż do restartu.
- Wykorzystując Service Control Manager - **CreateService** API
Sc.exe create nazwa binpath=ściezka parametry

- Zapis do \Device\PhysicalMemory

18. Omówić implementację pamięci wirtualnej za pomocą segmentacji.

Polega na przechowywaniu niektórych segmentów programu w pamięci dyskowej i sprowadzaniu ich do pamięci operacyjnej, gdy stają się potrzebne.

P					W	A
7	6 5	4	3	2	1	0

Deskryptor segmentu – bajt dostępu.

Obecność segmentu w pamięci operacyjnej wskazuje bit P(present).

Jeżeli wystąpi odwołanie do deskryptora z $P = 0$ to pojawia się wyjątek(przerwanie procesora), w takiej sytuacji system odszukuje wolny obszar pamięci (lub ją zwalnia), pobiera potrzebny segment do pamięci i powraca do realizowanego programu.

Aby racjonalnie zorganizować wymianę segmentów potrzeba mieć informację o statystykach wykorzystania poszczególnych segmentów, do zbierania informacji pomocny jest bit A(access). Ustawiany jest on gdy do rejestru segmentowego zostanie wpisany selektor wskazujący rozpatrywany deskryptor. System operacyjny (nie procesor) zeruje go w obserwowanych segmentach oraz okresowo odczytuje jego wartość i wykonuje zerowanie. Jeżeli odczytano kilkakrotnie 0, znaczy to iż segment jest nieużywany.

Istotny w implementacji jest również bit W, system może początkowo przypisać wszystkim segmentom atrybut „tylko do odczytu”(W=0), każda próba zapisu do danego segmentu wywoła wyjątek, może on sprawdzać „legalność” operacji i jeżeli zdecydujemy się na zapis ustawiamy bit W.

Daje to informacje o tym iż zawartość segmentu została zmieniona – jeżeli istnieje potrzeba usunięcia go z pamięci należy go przepisać na dysk, w przeciwnym wypadku zawartość segmentu w pamięci i na dysku jest jednakowa i nie ma potrzeby kopiowania go.

19. Jakie zalety ma realizacja pamięci wirtualnej za pomocą stronicowania w porównaniu z realizacją za pomocą segmentacji?

Segmentacja:

- deskryptor zawiera adres bazowy i rozmiar.
- segmenty różnych rozmiarów i słabo to systemowi przerzucać, pamięć się fragmentuje

Stronicowanie:

- zamiast adresy bazowego wpis w katalogu stron i indeks wpisu (deskryptor = adres fizyczny katalogu stron + indeks w tablicy stron + offset)

- odwołanie przez katalog stron -> tablica stron -> adres fizyczny (działa jak segregator z zakładkami segregator -> zakładka -> strona)
- każdy proces ma swój katalog stron (segregator) w rejestrze CR3
- w deskrytorze bit P - czy segment jest w pamięci -> zwalnianie ostatnio nieużywanych stron i ładowanie potrzebnych z pamięci. Proces odwołując się do ramki wstawia tam 1, system operacyjny go zeruje i robi sobie jakieś statystyki.
- rejestr cieni - procesor zapisuje sobie adresy rzeczywiste ramek, żeby nie robić za każdy razem tej translacji.
- tablica ostatnich adresów - używana jeśli nowy adres różni się tylko offsetem
- segmenty składają się ze stałego rozmiaru, zazwyczaj 4kb. Łatwiej jest systemowi zarządzać wieloma małymi kawałkami niż jednym dużym.

Segmentacja powoduje fragmentację pamięci (uboczny skutek wymiany segmentów o różnych rozmiarach), co nie ma miejsca przy stronicowaniu.

Poszczególne procesy mają dostęp do wyłącznie swoich własnych obszarów pamięci (odwołania przechodzą przez tablicę transformacji adresów i kierowane są pod obszar pamięci procesu).

20. W jaki sposób w architekturze IA-32 została zrealizowana koncepcja rejestru bazowego i granicznego jako metody izolacji procesów?

Rejestr bazowy przechowuje najmniejszy dopuszczalny adres fizyczny pamięci, a rejestr graniczny zawiera rozmiar obszaru pamięci (w innych architekturach może to być największy dopuszczalny adres fizyczny).

W IA-32 koncepcja realizowana jest poprzez określenie adresu bazowego i rozmiaru segmentu w odpowiednich polach deskryptora segmentu.

bity 31 – 24 adresu bazowego	G D 0 V	wlk.segm. 19 – 16	+6
bajt dostępu	bity 23 – 16 adresu bazowego		+4
bity 15 – 0 adresu bazowego			+2
wielkość segmentu (bity 15 – 0)			+0

Deskryptor

Uzyskujemy dzięki temu obszar do którego ma dostęp konkretny proces i poza który nie może się odwołać, w takim wypadku procesor zgłosi wyjątek.

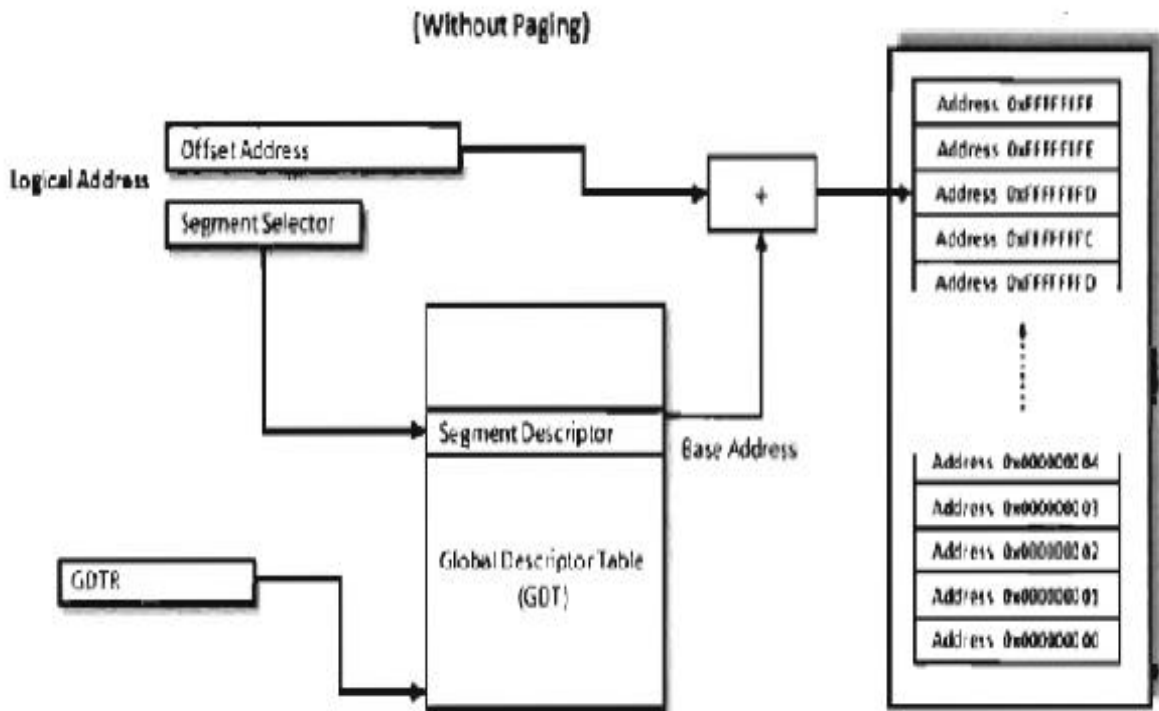
Wartości w rejestrach ustawia SO podczas przełączania kontekstu.

21. Opisz mechanizm wywołania funkcji systemowej na poziomie procesora (rejestru GDT)

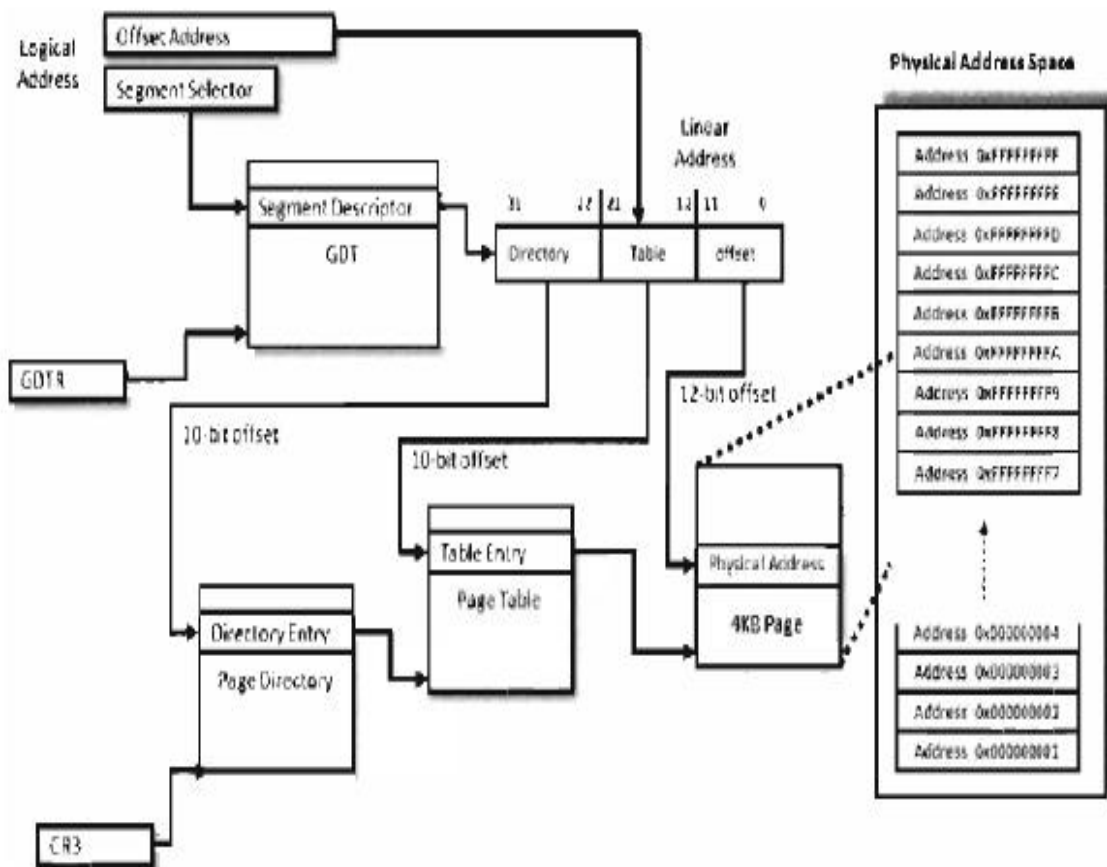


(WZIĘTE Z YETI, MOŻLIWE ŻE WYMAGA POPRAWY)

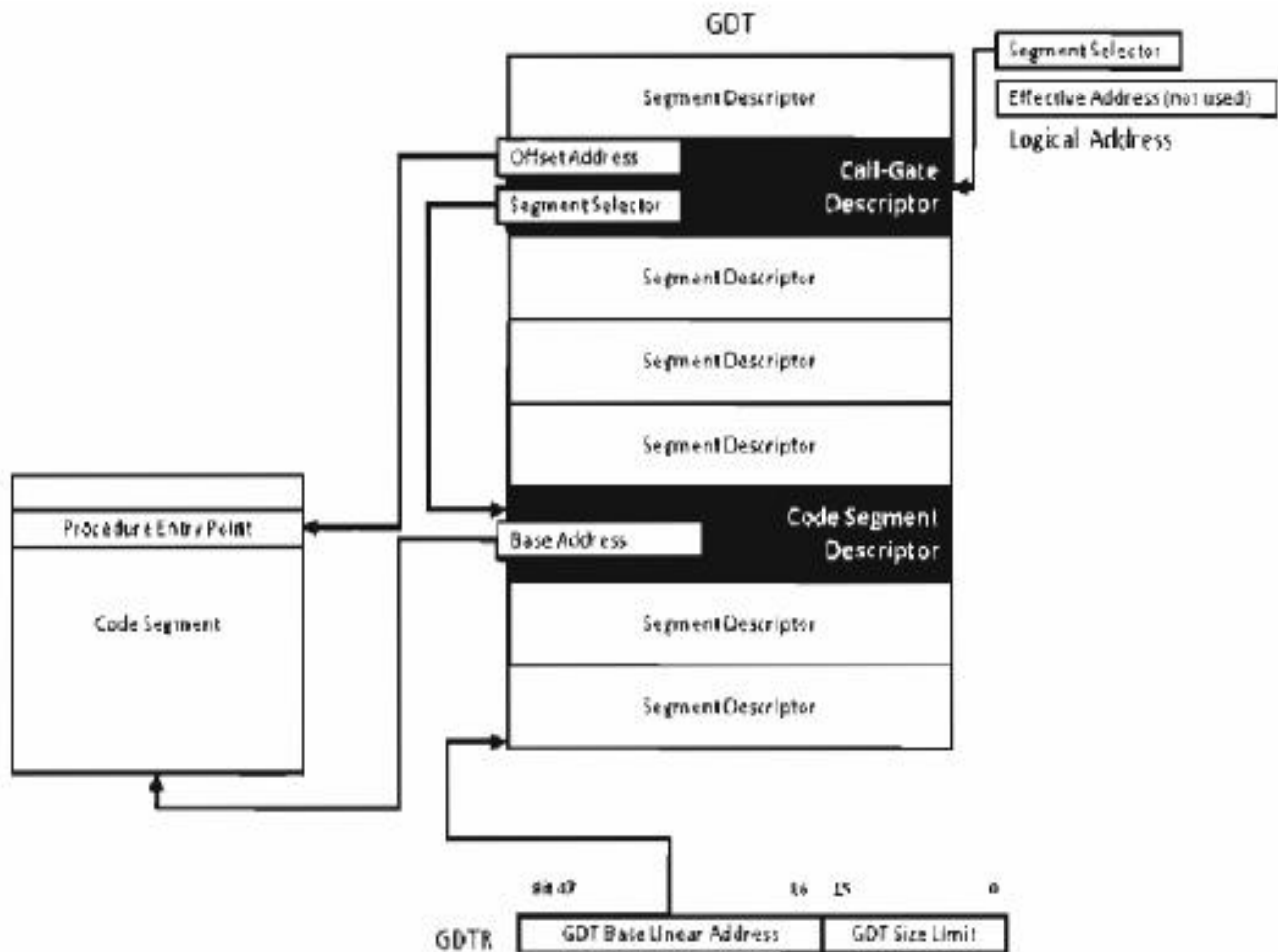
- Segmentowy model pamięci
 - Dzielimy pamięć na segmenty (każdy ma swój numer)
 - Adres: SELEKTOR (numer wskazujący segment) | PRZESUNIĘCIE (w danym segmencie)
 - Segmenty mogą mieć różną wielkość
- Pamięć, a tryby pracy procesora
 - Tryb rzeczywisty
 - Oparty o rejestry segmentowe
 - $\text{NUMER SEGMENTU (16 bitowy deskryptor)} \times 16 + \text{OFFSET (16 bitowy)} = \text{ADRES FIZYCZNY}$
 - Możliwość zaadresowania 1 MB
 - Pamięć jest uporządkowana
 - Każda aplikacja ma dostęp do całej pamięci (m. in. Wektorów przerwań i innych programów)
 - Można modyfikować adresy f-cji systemowych
 - Tryb chroniony z segmentacją
 - Procesy mają dostęp tylko do przydzielonych im segmentom



- Rejestr GDTR - adres początku tablicy deskryptorów
- Selektor wskazuje na deskryptor. Z niego pobierany jest adres segmentu. Po powiększeniu go od OFFSET mamy ADRES FIZYCZNY
- SELEKTOR - w nim jest wskaźnik na deskryptor i poziom uprzywilejowania segmentu wywołującego
 - [Segment selectors](#) contain a field called the requested privilege level (RPL). This allows a program to request a resource at a lower privilege level than it would otherwise use.
- DESKRYPTOR - w nim jest wskaźnik na segment, jego rozmiar oraz numer DPL (*Descriptor Privilage Level*)
 - DPL - wywołujący segment musi mieć poziom uprzywilejowania $DPL \geq CPL$ (Current Privilege Level)
 - Zawiera bajt dostępu, który pokazuje czy strona znajduje się na dysku i prz "obróbce" trzeba ją wczytać do pamięci operacyjnej (bit P)
- Stronicowanie
 - Adres z deskryptora jest dzielony na 3 części
 - Numer katalogu (10 bitów)
 - Numer tablicy (10 bitów)
 - Offset (12 bitów)



- CR3 - wskazuje na tablice katalogów stron
- Poziomy uprzywilejowania, dostanie się do niższego poziomu



- Wśród deskryptorów w tablicy znajdują się także bramy
- Brama ma w sobie selektor na inny deskryptor z niższym poziomem uprzywilejowania
- ADRES z 2 deskryptora | OFFSET z selektora wskazującego na bram = ADRES PROCEDURY SYSTEMOWEJ
- Każde zadanie ma 3 stopy - dla każdego poziomu uprzywilejowania
- Przerwania działają tak samo jak bramy
- Zalety stronicowanie
 - Brak fragmentacji zewnętrznej
 - W pamięci robią się dziury z których nikt już nie skorzysta
 - Strony są stałej wielkości i każda może być użyta
 - **Separacja przestrzeni adresowej dla każdego procesu = bezpieczeństwo**
 - Odzworowanie pliku dyskowego w przestrzeń adresową procesu
 - **Powiększenie pamięci fizycznej**
- wywoływanie podprogramów na wyższych poziomach uprzywilejowania możliwe jest tylko za pośrednictwem specjalnych deskryptorów zwanych furtkami wołania (ang. call gates) – furtki

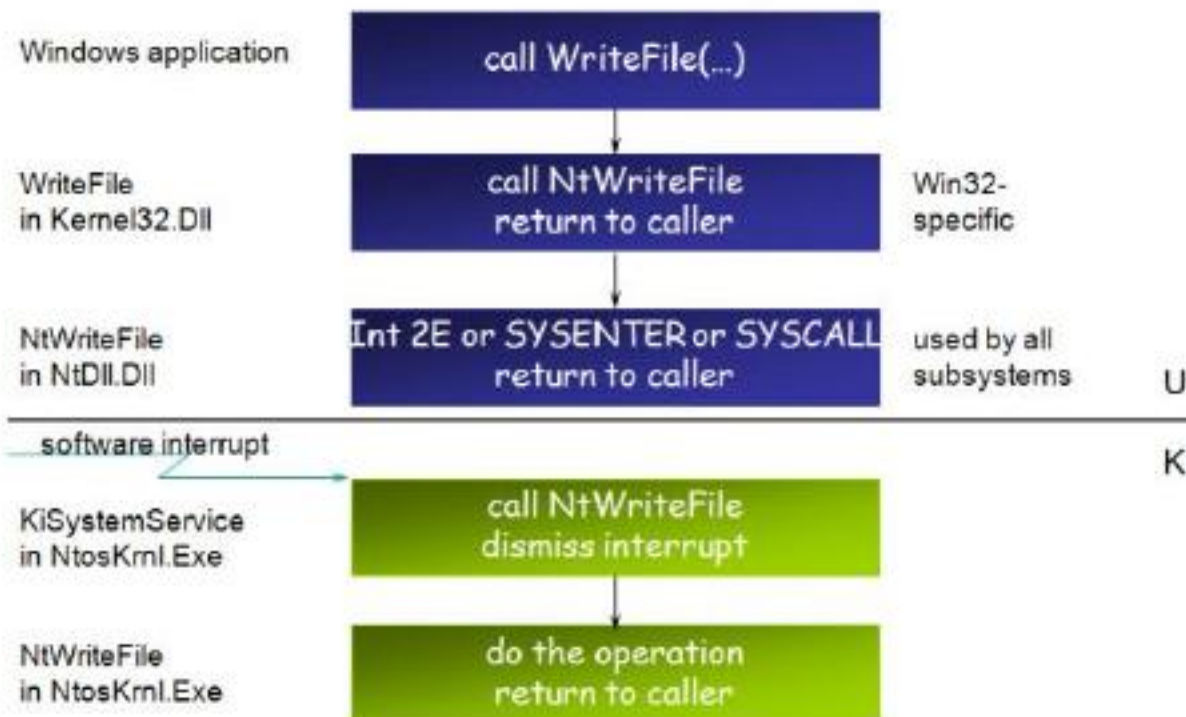
wołania, przechowywane w tablicy deskryptorów, stanowią jedyne legalne punkty wejścia do podprogramów;

- każda furtka ma przypisany pewien poziom uprzywilejowania; furtka wołania jest dostępna dla wszystkich programów uprzywilejowanych nie gorzej niż poziom furtki; wywołanie podprogramu za pośrednictwem furtki wołania polega na wykonaniu rozkazu CALL typu FAR: pole selektora tego rozkazu wskazuje furtkę, a offset jest ignorowany;
- pole DPL furtki określa konieczny stopień uprzywilejowania dla programu, który korzysta z furtki; zatem furtka może być użyta tylko wówczas, gdy $DPL \leq CPL$, gdzie CPL oznacza bieżący poziom uprzywilejowania;

22. Opisz mechanizm wywołania funkcji systemowej na poziomie jądra systemu operacyjnego Windows XP

(WZIĘTE Z YETI, MOŻLIWE ŻE WYMAGA POPRAWY)

- Wywołanie funkcji Windowsowej np. WriteFile. Przez systemowe biblioteki dochodzi to do NTDLL
- NTDLL wykonuje funkcję Int2E, SYSENTER lub SYSCALL, która powoduje tzw. "przerwanie programowe"
- Systemowy dispatcher obsługuje przerwanie
 - Szuka odpowiedniego wpisu w tablicy przerw
 - Wpis zawiera numer tablicy i numer indeksu w tej tablicy - tam znajduje się API funkcji systemowej
 - Po takim odwołaniu funkcja zostaje uruchomiona



23. Przedstaw działanie procesu planisty w systemie Windows XP

Istnieją 32 kolejki wątków oznaczające priorytety (31 najwyższy), 31-16 to real time, kolejne 15 zmiennych, i najniższy zarezerwowany dla wątku podstawowego. planista zawsze wybiera wątki o najwyższym priorytecie zgodnie z algorytmem 'round robin'. Po upływie kwantu czasu planista określa czy należy zmienić priorytet aktualnie wykonywanemu wątkowi, następnie rozpoczyna wykonywanie kolejnego wątku.

Każdy wątek ma dwa priorytety: bazowy i bieżący. Nowy wątek dziedziczy priorytet procesu (bazowy). Procesy zużywające dużo zasobów procesowa przenoszona są do kolejek o niższym priorytecie.

Zwiększenie wartości priorytetu:

- zakończenie operacji IO (+X w zależności od sterownika)
- wątek GUI dostaje focusa +2
- proces gotowy do działania nie działał od 4s (=15 i podwojenie kwantu)
- dostał event lub opuszczenie semafora +1

Podczas zmian wartości procesu jego priorytet nie może przekroczyć 15, co cykl wartości są obniżane o 1 aż do bazowego.

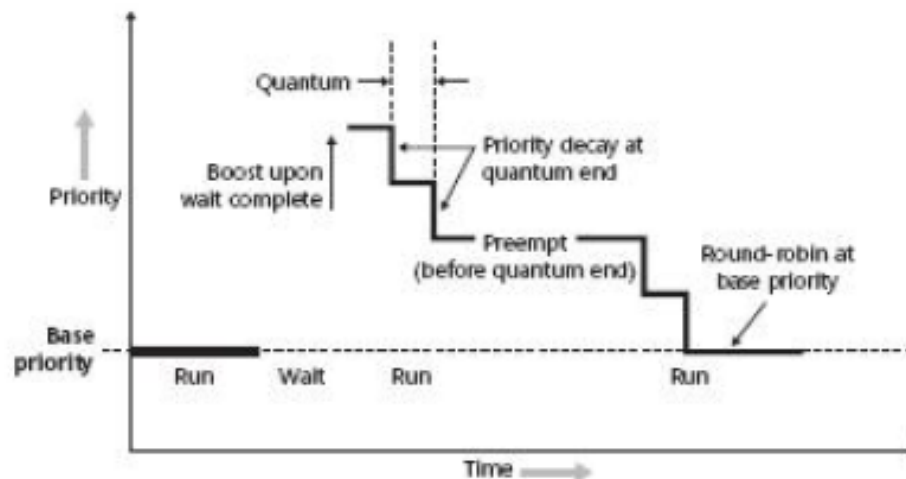


FIGURE 5-23 Priority boosting and decay

Taktyki szeregowania:

- dobrowolne przełączenie - gdy wątek wywoła coś jak wait()

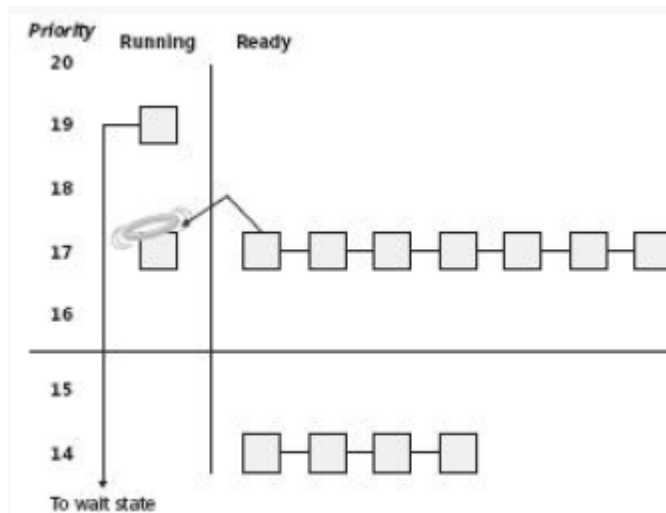


FIGURE 5-18 Voluntary switching

- koniec kwantu - planista sprawdza czy nie wpierdolił za dużo i mu nie obniżyć

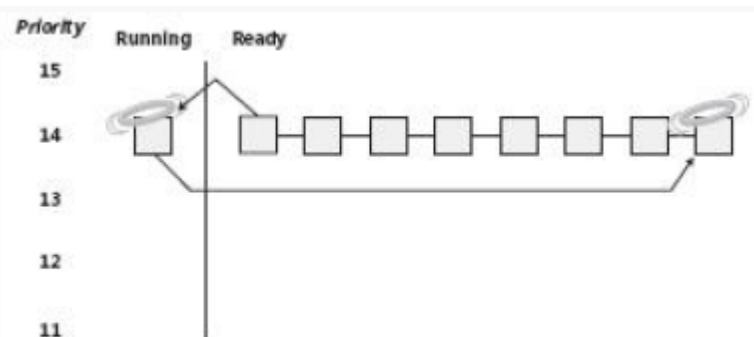
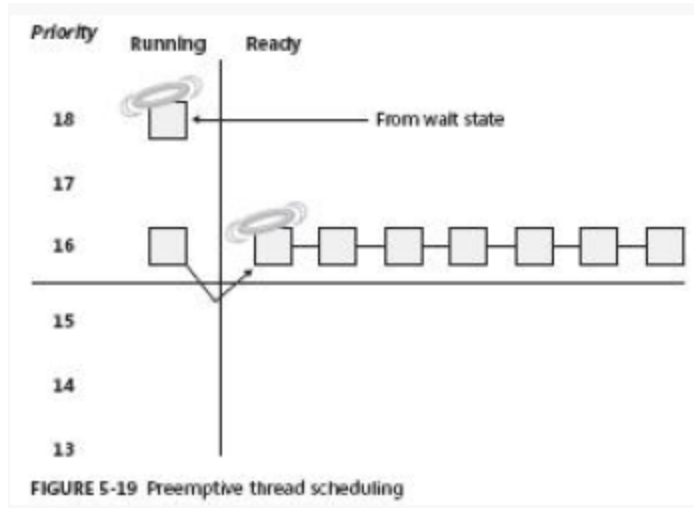


FIGURE 5-20 Quantum end thread scheduling

- do kolejki wpadł wątek o wyższym priorytecie -> przerywany jest akt wykonywany o niższym



24. Wymień i scharakteryzuj metody łączenia kodu bibliotecznego z kodem programu

- Linkowanie statyczne polega na dołączeniu odpowiedniego kodu funkcji do programu na etapie kompilacji.
- Linkowanie dynamiczne polega na dołączaniu do programu jedynie odpowiednich odwołań do funkcji bibliotecznych.
Sam kod funkcji nie jest dołączany do programu na etapie kompilacji.
Przy uruchomieniu programu, biblioteka jest natomiast ładowana do pamięci i udostępnia odpowiednie funkcje pracującemu programowi.

Podstawową zaletą linkowania dynamicznego jest to, że jest wymagany jeden egzemplarz biblioteki w pamięci dla wszystkich programów które z niej korzystają. Wadą jest to, że program linkowany dynamicznie jest zależny od dostępności zewnętrznym plików (bibliotek) i nie będzie działał poprawnie jeżeli ich nie znajdzie.

Metoda dynamiczna może być w 2ch wariantach: RTL(run) i LTL(load) w czasie wykonania i w czasie ładowania.

RTL: LoadLib("dsd.dll");

LTL: przy ładowaniu programu system sam nam załaduje

25. Opisz budowę pliku PE

Jest to wspólny format dla plików wykonywalnych i bibliotek dll w systemie windows.

Zawiera liste funkcji ktore daje i ktore chce.

Plik składa się z:

- sygnatury "PE\0\0"
- małego programu dosowego wypisującego ze program wymaga systemu windows.
- nagłówek COFF (typ procesora, liczba sekcji, rozmiar opcjonalnego nagłówka, bit exec czy dll)
- opcjonalny nagłówek
- katalogi danych (importy, exporty)
- nagłówki sekcji
- sekcje

26. Przedstaw kolejne kroki w komunikacji pomiędzy aplikacją użytkową a sprzętem w systemie Windows

Aplikacja użytkownika wywołuje funkcję z Windows API, odwzorowywaną na funkcje z ntdll.dll.

Następnie musi zostać wywołana funkcja na poziomie kernela (wyższy poziom uprzywilejowania).

Odbywa się to za pomocą mechanizmu deskryptora bram.

Żądanie trafia do warstwy HAL (hal.dll), która już ogarnia komunikację ze sprzętem.

27. Opisz proces ładowania programów do pamięci operacyjnej

- ładowanie bezwzględne

cały kod programu musi być w pełni określony w chwili ładowania go do pamięci;
programista może posługiwać adresami fizycznymi lokacji pamięci, lub te adresy te mogą
wyznaczone przez kompilator (assembler) i linker;
niezbędna jest jednak znajomość adresu początkowego obszaru pamięci, do którego zostanie
wpisany program;

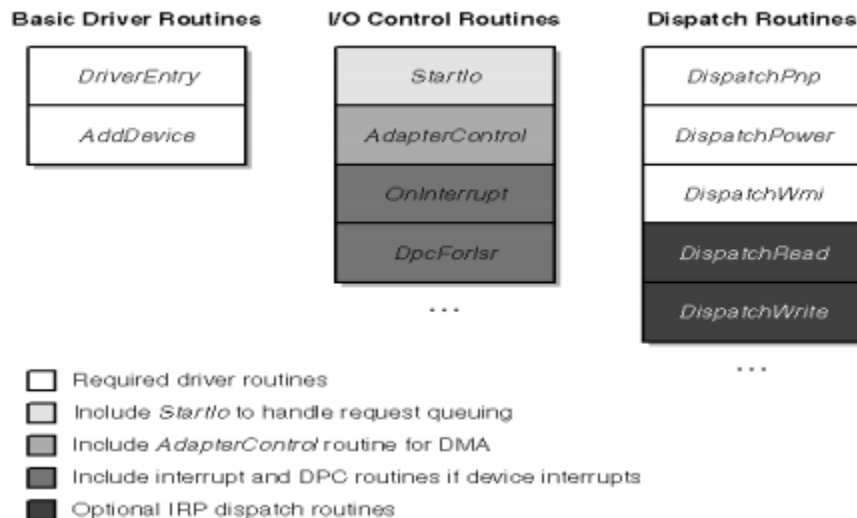
- ładowanie relokowalne

decyzję o położeniu programu w pamięci podejmuje się podczas ładowania
w takim przypadku program można umieścić w dowolnym obszarze pamięci;

- ładowanie dynamiczne

jest charakterystyczne dla współczesnych wielozadaniowych systemów operacyjnych, w których
stosuje się pamięć wirtualną;
wówczas poszczególne fragmenty programu mogą być rozproszone w pamięci operacyjnej, a
część z nich może być przechowywana na dysku;
ten sam fragment programu, jeśli zostanie ponownie przepisany z dysku do pamięci operacyjnej,
może pracować w obszarze o innych adresach fizycznych;
realizacja ładowania dynamicznego wymaga więc skomplikowanych sprzętowych mechanizmów
transformacji adresów (zob. stronicowanie).

28. Omów budowę podstawowego szkieletu sterownika urządzeń WDM w systemie Windows.



Procedury obowiązkowe

- **DriverEntry:** Funkcja inicjalizująca działanie sterownika. Jej zadanie to wskazanie SO, gdzie znajdują się niezbędne funkcje (np. AddDevice). W praktyce oznacza to wskazanie SO przez sterownik, które funkcje może użytkownik (a raczej jego aplikacja) wykonać.

- **AddDevice:**

- Tworzy obiekt urządzenia (device object) oraz instancję rozszerzenia urządzenia (device extension)
- Rejestruje jedno lub więcej interfejsów urządzeń ---Register one or more device interfaces to let applications know about your device's existence
- Wpisuje device extension i flagi (flags member) do obiektu urządzenia (device object).
- Dołącza device object do stosu urządzeń (device stack).

A NIE MUSI BYĆ JESZCZE Unload? Na wykładach jest.

/*

- An initialization routine The I/O manager executes a driver's initialization routine, which is typically named **DriverEntry**, when it loads the driver into the operating system. The routine fills in system data structures to register the rest of the driver's routines with the I/O manager and performs any global driver initialization that's necessary.
- An **add-device** routine A driver that supports Plug and Play implements an add-device routine. The PnP manager sends a driver notification via this routine whenever a device for which the driver is responsible is detected. In this routine, a driver typically allocates a device object (described later in this chapter) to represent the device.
- A set of **dispatch routines** **Dispatch** routines are the main functions that a device driver provides. Some examples are open, close, read, and write and any other capabilities the device, file system, or network supports. When called on to perform an I/O operation, the I/O manager

generates an IRP and calls a driver through one of the driver's dispatch routines.

*/

29. Ile stosów ma procedura w trybie chronionym? Wyjaśnij dlaczego tak się dzieje.

Dana procedura wykonywana jest przez wątek. Każdy wątek posiada 2 stosy – po jednym w trybie jądra i użytkownika. Powodem tego jest bezpieczeństwo – by nic co działa w trybie użytkownika nie miało dostępu do danych z trybu jądra.

30. Opisz proces ładowania sterowników dla urządzeń PnP.

- podczas startu system ładuje sterownik magistrali i przepytuje każde urządzenie na portach PnP, tworzy i ładuje jego device object.
- każde urządzenie ma swój unikalny łańcuch i zostawia ślad w rejestrze - adres drivera
- system szuka najpierw czy zna już to urządzenie
- pierwszy sterownik urządzenia może rejestrować kolejne dla podurządzeń (urządzenia o wielu funkcjach)

31. Wymień i opisz sposoby komunikacji aplikacji użytkowej ze sterownikiem WDM.

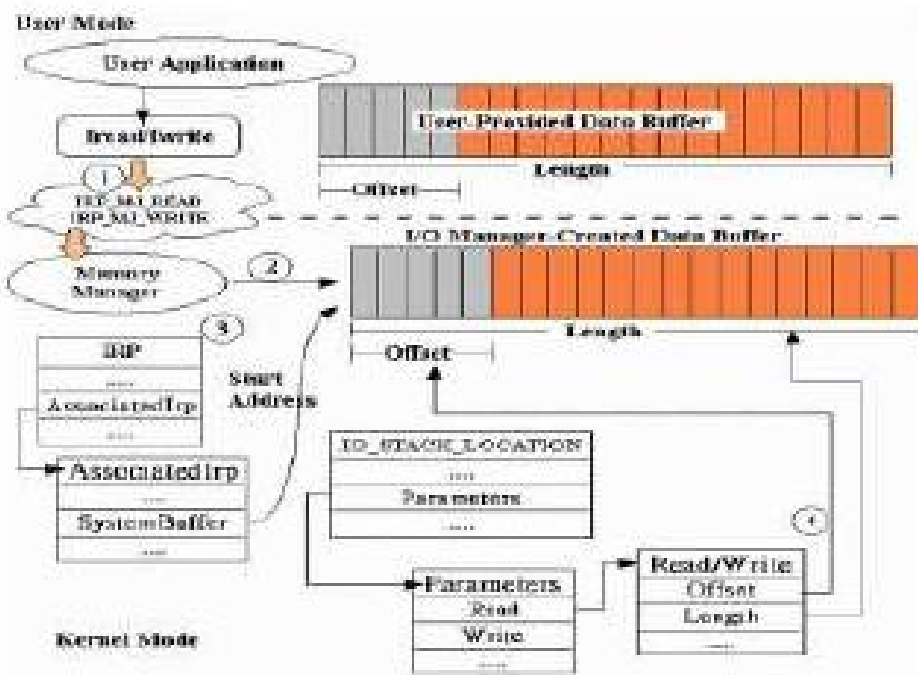
(nie wiem czy tu o to chodzi)

- buforowane - system kopiuje pamięć z user do kernem mode
- bezpośrednie - kopiowanie tylko dane strony

(WZIĘTE Z YETI, MOŻLIWE ŻE WYMAGA POPRAWY)

- Buforowany

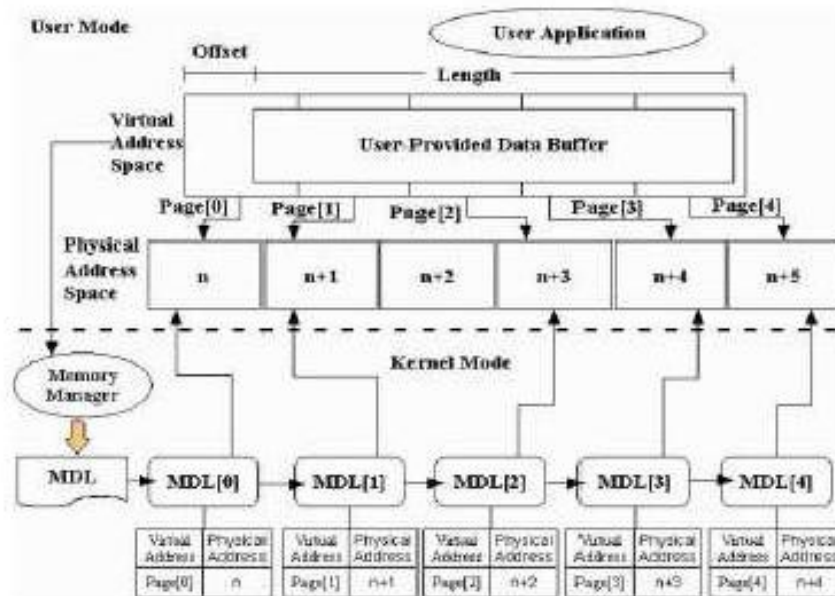
Buforowane operacje I/O



The operating system creates a nonpaged system buffer, equal in size to the application's buffer. For write operations, the I/O manager copies user data into the system buffer before calling the driver stack. For read operations, the I/O manager copies data from the system buffer into the application's buffer after the driver stack completes the requested operation.

- Bezpośredni

Direct I/O



- The operating system locks the application's buffer in memory. It then creates a memory descriptor list (MDL) that identifies the locked memory pages, and passes the MDL to the driver stack. Drivers access the locked pages through the MDL.
- **Własny (neither)**

32. Omów typowe działanie systemu operacyjnego przy błędach strony (page fault handling).

Znalezienie w pamięci strony rzadko używanej i jej podmiana ze stroną żadaną.

(WZIĘTE Z YETI, MOŻLIWE ŻE WYMAGA POPRAWY)

When a page fault occurs, the hardware cannot do anything else with the instruction that caused the page fault and thus it must transfer control to an operating system routine (this is the page fault handler). The page fault handler must then decide how to handle the page fault. It can do one of two things:

- It can decide the virtual address is just simply not valid. In this case, Windows will report this error back by indicating an exception has occurred (typically STATUS_ACCESS_VIOLATION)
- It can decide the virtual address is valid. In this case, Windows will find an available physical page, place the correct data in that page, update the virtual-to-physical page translation mechanism and then tell the hardware to retry the operation. When the hardware retries the operation it will find the page translation and continue operations as if nothing had actually happened.

Hardware generates a page fault for page accesses where:

- the page corresponding to the requested address is not loaded in memory.
- the page corresponding to the memory address accessed is loaded, but its present status is not updated in hardware.

The closely related exception known as the **protection fault** is generated for page accesses where:

- the page is not part of the program, and so is not mapped in program memory.
- the program does not have sufficient privileges to read or write the page.
- the page access is legal, but it is mapped with [demand paging](#).

Protection fault can also be generated for many other invalid accesses not related to paging.

A *hard page fault* must be resolved by reading page contents from the page's *backing store*, which is either the system paging file or a memory-mapped file created by the process. A *soft page fault* can be resolved without accessing the backing store. A soft page fault occurs when:

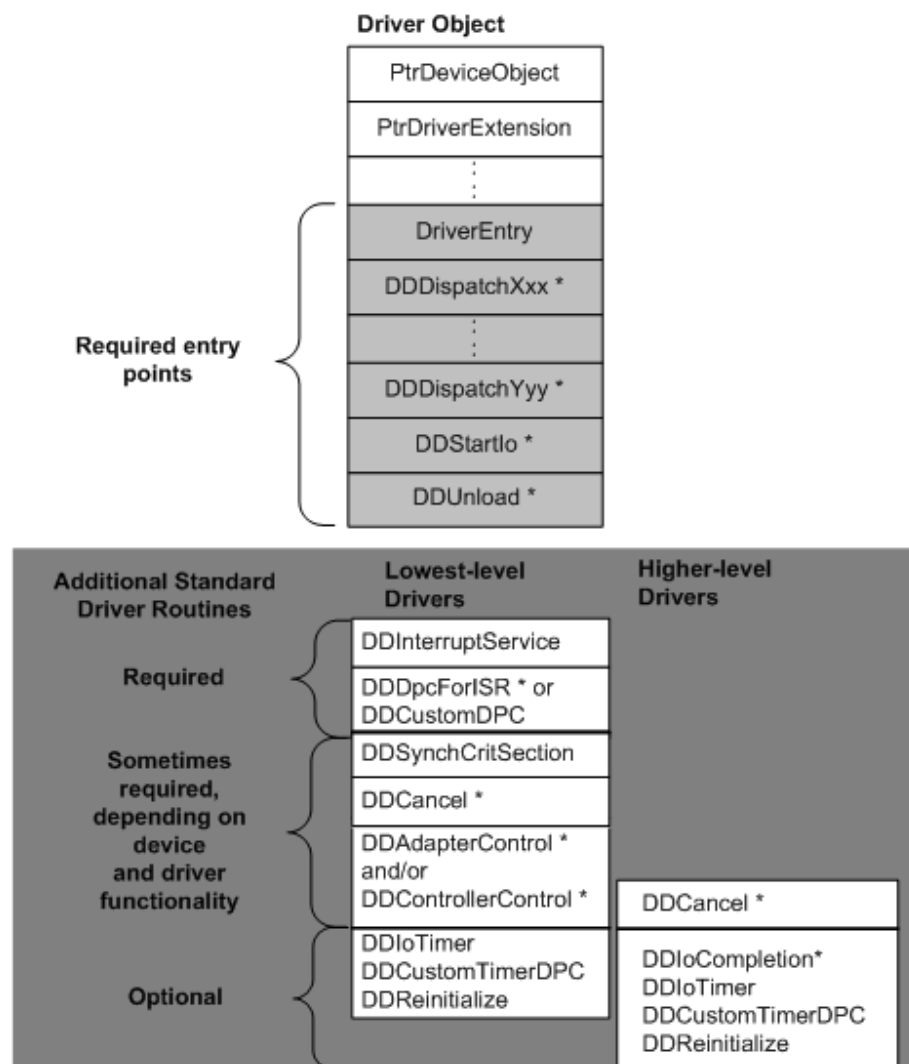
- The page is in the working set of some other process, so it is already resident in memory.
- The page is in transition, because it either has been removed from the working sets of all processes that were using the page and has not yet been repurposed, or it is already resident as a result of a memory manager prefetch operation.
- A process references an allocated virtual page for the first time (sometimes called a *demand-zero fault*).

33. Na czym polega zagadnienie DLL-hell. Wymień sposoby pozwalające na uniknięcie tego problemu.

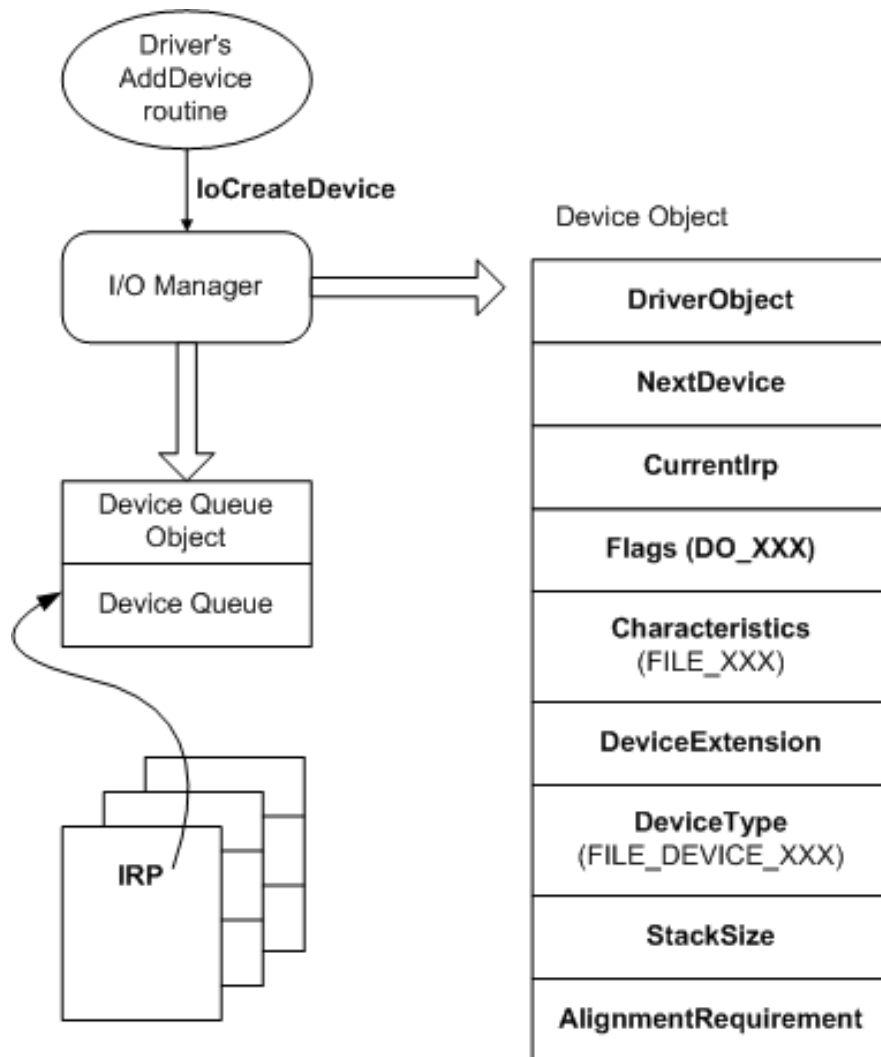
Polega na tym, że aplikacje użytkownika do działania wymagają konkretnych wersji dll. W starszych systemach Windows instalowanie aplikacji powodowało zazwyczaj nadpisanie obecnej w systemie wersji biblioteki dll. Sposób: aplikacja trzyma swoje dll w swoim folderze (tam loader zagląda najpierw), problem polega na duplikowaniu tych samych bibliotek. Można do dll dodać numer wersji, aby uniknąć duplikowania.

34. Wymień trzy podstawowe struktury systemowe wykorzystywane przy budowie i działaniu sterowników urządzeń i podaj ich znaczenie

- DRIVER OBJECT – reprezentuje obraz załadowanego sterownika.
- DEVICE OBJECT – reprezentuje urządzenie fizyczne lub logiczne w systemie i opisuje jego charakterystykę
- IRP (I/O Request Packet) – paczki z danymi, przesyłane między sterownikami.



Jak jest tworzony device object:



35. Podaj różnicę pomiędzy zadaniami (jobs), procesami, wątkami i włóknami.

Jobs to zbiór wątków który jest kolejkowany tak samo jak pojedynczy wątek,

Włókno to lekki wątek pozwalający na szeregowanie wątków po stronie użytkownika, bez ingerencji kernela.

Procesy mają oddzielną przestrzeń adresową i kontekst.

36. Jaka jest rola mechanizmu DPC i na czym on polega ?

- Odroczone wywołanie procedury (DPC) jest wykorzystane do wykonania kodu przy przejściu z wyższego poziomu IRQL na niższy
 - Sterownik (proces jądra) kolejkuje żądania,
 - wykonanie żądań z kolejki następuje w momencie zakończenia pracy wszystkich przerwań o wyższym IRQL

- Wykorzystane jest do generowania timeout'ów oraz przełączania wątków. Dzięki temu mechanizmowi operacje krytyczne (np. funkcje sterowników sprzętowych) są wykonywane szybciej niż operacje, które nie muszą być wykonywane od razu.

A **Deferred Procedure Call (DPC)** is a Windows [operating system](#) mechanism which allows high-priority tasks (e.g. an [interrupt handler](#)) to defer required but lower-priority tasks for later execution. This permits device drivers and other low-level event consumers to perform the high-priority part of their processing quickly, and schedule non-critical additional processing for execution at a lower priority. DPCs are implemented by DPC objects which are created and initialized by the [kernel](#) when a [device driver](#) or some other kernel mode program issues requests for DPC. The DPC request is then added to the end of the system-wide DPC [queue](#). DPCs have three priority levels: low, medium and high. By default, all DPCs are set to medium priority. When the processor drops to an [IRQL](#) (interrupt request level) of Dispatch/DPC level, it checks the DPC queue for any pending DPCs and executes them until the queue is empty or some other interrupt with a higher IRQL occurs.

For example, when the clock interrupt is generated, the clock interrupt handler generally increments the counter of the current thread to calculate the total execution time of that thread, and decrements its [quantum](#) time remaining by 1. When the counter drops to zero, the thread scheduler has to be invoked to choose the next thread to be executed on that processor and dispatcher to perform a context switch. Since the clock interrupt occurs at a much higher IRQL, it will be desirable to perform this thread dispatching which is a less critical task at a later time when the processor's IRQL drops. So the clock interrupt handler requests a DPC object and adds it to the end of the DPC queue which will process the dispatching when the processor's IRQL drops to DPC/Dispatch level.

37. Jaka jest rola mechanizmu APC i na czym on polega ?

(WZIĘTE Z YETI, MOŻLIWE ŻE WYMAGA POPRAWY)

An asynchronous procedure call (APC) is a function that executes asynchronously in the context of a particular thread. When an APC is queued to a thread, the system issues a software interrupt. The next time the thread is scheduled, it will run the APC function. An APC generated by the system is called a kernel-mode APC. An APC generated by an application is called a user-mode APC. A thread must be in an alertable state to run a user-mode APC.

Each thread has its own APC queue. An application queues an APC to a thread by calling the QueueUserAPC function. The calling thread specifies the address of an APC function in the call to QueueUserAPC. The queuing of an APC is a request for the thread to call the APC function.

When a user-mode APC is queued, the thread to which it is queued is not directed to call the APC function unless it is in an alertable state. A thread enters an alertable state when it calls the SleepEx, SignalObjectAndWait, MsgWaitForMultipleObjectsEx, WaitForMultipleObjectsEx, or WaitForSingleObjectEx function. If the wait is satisfied before the APC is queued, the thread is no longer in an alertable wait state so the APC function will not be executed. However, the APC is still queued, so the APC function will be executed when the thread calls another alertable wait function.

APC wykorzystywane jest przy asynchronicznej komunikacji między aplikacją użytkownika a sprzętem.

38. Wyjaśnij następujące pojęcia MBR, VBR, DDK, IDT, GDTR.

- **VBR** – Volume Boot Record. Podobny do MBR, ale znajduje się już na konkretnej partycji (na początku) i zawiera program, który powinien rozpocząć ładowanie systemu operacyjnego.
- **MBR** – Master Boot Record. Jest to pierwszy sektor na dysku zawierający specjalny program, który jest odczytywany przy starcie komputera do pamięci RAM i którego zadaniem jest przeszukanie tablicy partycji w celu przejścia do VBR.
- **DDK** – Driver Development Kit. Zestaw narzędzi pozwalających na tworzenie sterowników urządzeń (i nie tylko) na pewną platformę.
- **IDT** – Interrupt Descriptor Table. Jest to tablica deskryptorów znajdująca się w pamięci RAM, w której zapisane są selektory, które z kolei mają indeksować tablicę GDT, która wskazuje segment, w jakim znajduje się procedura obsługi przerwania.
- **GDTR** – Global Descriptor Table Register. Jest to 48-bitowy rejestr wskazujący położenie w pamięci tablicy GDT (32 bity) oraz limit rozmiaru tej tablicy (16 bitów).

39. Wymień 5 różnych systemów plików.

fat32 - listowy plik do 4GB, dysk do 2TB

reiserfs - journaling, wydajny dla małych plików,

ntfs - hybryda listowej i indeksowej metody (lista kolejnych segmentów pliku zawierających rozmiar ciągłego obszaru zajmowanego przez fragment pliku)

xfs - 64bitowy gwarantowanie pasma dostępu do pliku

ext3-metoda indeksowa, journaling, można montować jako ext2(narzędzie tune2fs), do 32TB, mniej obciąża procesor niż reiser i xfs

40. Omówić mechanizm przełączania stosu używany w trakcie przyjmowania przerwania.

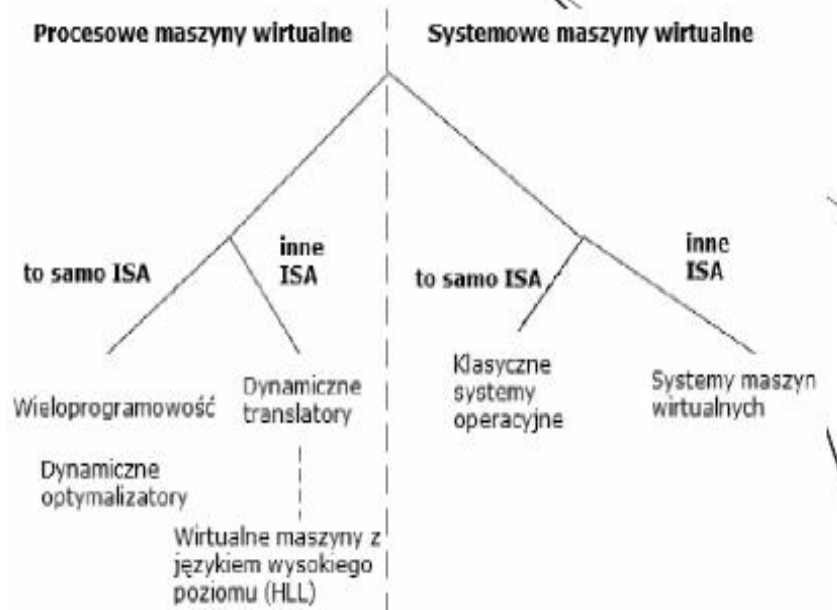
Mechanizm ten jest identyczny jak w przypadkuwołania funkcji za pomocą furetek. Potrzeba jego użycia zachodzi, gdy program wołany jest bardziej uprzywilejowany niż program wołający.

Przebiega następująco:

- nowe wartości rej. SS i ESP przechowywane są w TSS (Task State Segment)
- są one ładowane do SS i ESP, a stare są zachowywane na nowym stosie
- ze starego stosu na nowy kopiowane są parametry podprogramu
- na stosie zapisywany jest ślad (zawartość CS i EIP).

41. Przedstawić klasyfikację maszyn wirtualnych i opisać ich charakterystyki

Klasyfikacja VM



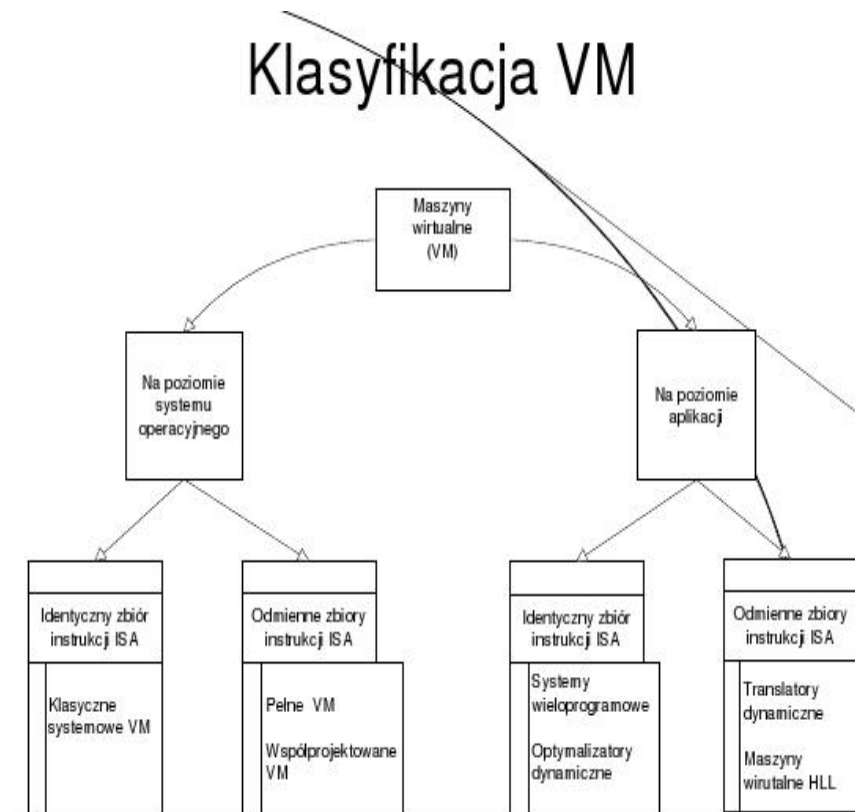
Maszyny wirtualne to m.in.:

- * interpretery, szczególnie interpretery kodu bajtowego
- * kompilatory JIT
- * emulatory rzeczywiście istniejącego sprzętu, np. emulatory konsol

Maszyny wirtualne można podzielić na maszyny wirtualne procesowe i systemowe. VM na poziomie procesu dostarcza **ABI (Application Binary Interface – rozdziela procesy od systemu operacyjnego)** dla aplikacji, a systemowa dostarcza zbioru **ISA (Instruction Set Architecture – rozdziela sprzęt od pozostałej części systemu)** dla systemu operacyjnego i aplikacji.

Procesowe maszyny wirtualne działają jako normalny proces w systemie, są z nim tworzone i usuwane. Ich celem jest przede wszystkim uniezależnienie programowania od platformy, by programy były przenośne mimo różnic sprzętowych (lub też programowych) w maszynach. Przykładem procesowej maszyny wirtualnej może być Java Virtual Machine. Systemowe maszyny wirtualne zezwalają na współdzielenie zasobów sprzętowych między maszyny wirtualne, z których każda może być uruchomiona z innym systemem operacyjnym. Przykładem jest VirtualBox. Wadą tego rozwiązania jest

to, że wydajność systemu zainstalowanego na maszynie wirtualnej jest niższa.



42. Opisz strukturę bio zdefiniowaną w <linux/bio.h>

Reprezentuje aktywne operacje IO w postaci listy ciągłych obszarów w pamięci (segmentów). Bufory nie muszą zajmować ciągłej przestrzeni w pamięci (scatter-gather I/O)

Głównymi elementami struktury są:

- bio_io_vec - tablica wektorów bio_vec - opisuje ona buffor.
- bio_vec - wektor opisujący segmenty.
- Informacje o urządzeniu do którego się odnosi
- Informacje o typie operacji (read/write)
- licznik użyc

Struktura nie zawiera informacji o stanie buffora.

43. Porównaj poznane schedulery dla urządzeń blokowych I/O w systemie Linux.

Linus elevator - domyślny dla 2.4, scala pobliskie żądania odczytu. (występuje problem write starving reads, głodzenie innych przy wielu operacjach w jednej części dysku)

deadline scheduler - każde żądanie ma deadline (500ms odczyt, 5s zapis). Trzymana dodatkowa kolejka posortowana po tych czasach (osobno dla odczytu i zapisu) i gdy operacja przekroczy deadline jest brana z tej kolejki.

anticipatory scheduler - heurystycznie przewiduje operacje wg. statystyk każdego procesu.

cfq - domyślny dla 2.6, osobna kolejka dla każdego procesu, pobieranie round-robinem.
noop - w ogóle brak sortowania.

44. Czym charakteryzuje się problem „write starving reads” i gdzie występuje ?

Problem związany z kolejkowaniem żądań IO przez linux elevator (i nie tylko)(domyślny dla jądra 2.4).

Zapis asynchroniczny, odczyt synchroniczny.

Zapisy w jednej części dysku blokują odczyty w innej (aplikacja czeka na dane).

Deadline zapobiega częstotliwości zapobiega temu problemowi a anticipatory robi to świetnie.

45. Przedstaw strukturę i-node dla systemu plików ext3.

Każdy i-węzeł zawiera:

- Opis położenia pliku na dysku i rozmiar pliku
- Identyfikator właściciela pliku,
- Typ pliku (pliki zwykłe, katalogi, ...),
- Prawa dostępu do pliku (właściciel pliku, grupowy właściciel pliku, pozostali użytkownicy),
- Czas dostępu do pliku (czas ostatniej modyfikacji, czas ostatniego dostępu, czas ostatniej modyfikacji i-węzła pliku),
- Liczba dowiązań do pliku,
- Liczba procesów, które otworzyły dany plik,
- Inne,
- można zauważyć, że i-węzeł nie zawiera nazwy pliku

46. Podaj blok oraz offset, gdzie znajduje się bajt 6 000 000 w systemie plików ext3. Przedstaw tok obliczania.

bezpośrednie - do 48kb

pośrednie raz - 4mb

pośrednie dwa razy - 4gb

pośrednie trzy razy 4tb

czyli nasz wymaga użycia podwójnie pośredniego (6 000 000 e <4mb, 4gb>)

Pierwszy bajt dostępny jest poprzez blok podwójnie pośredni i ma numer $48KB + 4MB = 49152 + 4194304 = 4\,243\,456$

każdy wpis w bloku podwójnie pośrednim to kolejne 4mb zaadresowane, czyli odczytujemy pierwszy wpis i nasz bajt jest $6\,000\,000 - 4\,243\,456 = 1\,756\,544$ względem niego

to teraz kolejny lvl - adresowanie raz pośrednie

każdy wpis to 4mb, czyli odczytujemy $1\,756\,544 / 4kb = 428$ wpis i nasz bajt jest $1\,756\,544 - 4kb * 428 = 3456$ względem niego

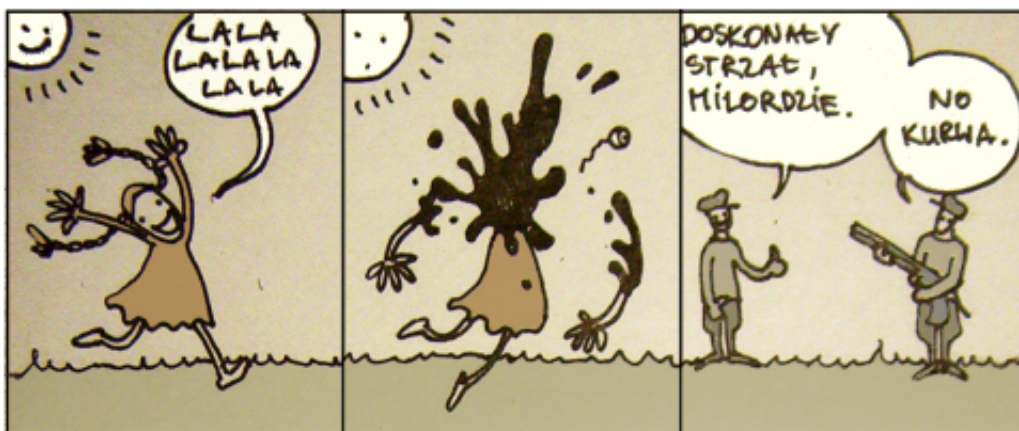
47. Przedstaw proces zamykania procesu w systemie Linux (do_exit()).

do_exit() nam robi z procesu zombie i informuje rodzica o zakończeniu (zostawia mu jakieś dane), jak rodzic je odczyta to wszystko jest zwalnia i proces umiera.

Bardziej szczegółowy opis:

- I. ustawia flagę PF_EXITING w polu flags w strukturze task_struct,
- II. del_timer_sync() – deaktywacja timera – w odróżnieniu od del_timer() – gwarantowane, że żaden nie jest zakolejkowany bądź uruchomiony nawet na innym procesorze,
- III. If BSD process accounting ins enabled, do_exit() calls acct_update_integrals() to write out accounting information,
- IV. exit_mm() – zwolnienie mm_struct, gdy nie jest ona wykorzystywana przez inny proces (współdzielenie) – kernel usuwa ją,
- V. exit_sem() – gdy proces oczekuje na semafor IPC – jest usuwany z kolejki,
- VI. exit_files(), exit_fs() – dekrementacja licznika użycia obiektów skojarzonych z deskryptorami plików oraz danych dotyczących systemu plików (tak samo jak w przypadku exit_mm – gdy licznik jest 0 – obiekt jest usuwany),
- VII. Ustawienie exit_code w task_struct na kod przekazany do instrukcji exit() bądź kod określający, że proces został przerwany przez kernela. Pole exit_code przeznaczone jest do opcjonalnego pobrania przez proces rodzica.
- VIII. exit_notify() – informowanie procesu rodzica o zakończeniu działania, przypisanie dzieciom nowego rodzica – forget_original_parent() – find_new_reaper() (procesu znajdującego się w tej samej grupie procesów bądź procesowi init), ustawienie exit_state w task_struct na EXIT_ZOMBIE,
- IX. schedule() – przełączenie na inny proces – z uwagi, że proces nie jest już poddawany harmonogramowaniu jest to ostatnia funkcja przez niego wykonywana, do_exit() nigdy nic nie zwraca

48. Co to jest proces zombie i jak należy z nim postępować?



- I. po wykonaniu funkcji do_exit() proces przechodzi do stadium EXIT_ZOMBIE, pozostaje zaalokowana jeszcze pamięć w postaci: stosu kernela, struktury thread_info i task_struct. Informacje te po odebraniu przez proces rodzica (wait()) bądź po oświadczeniu, że nie jest on nimi zainteresowany pamięć ta jest

zwalniana i udostępniana systemowi do ponownego użycia.

W skrócie, aby dobić zombie musimy poprosić o to jego rodzica lub go (rodzica) zabić.

Zasoby po procesie zombie zwalnia funkcja `release_task()`;

II. Znajdowanie procesów zombie `ps - el | grep 'Z'`

49. W jaki sposób w systemie Linux tworzony jest nowy proces?

`fork()` nam robi kopie procesu i ustawia mu PPID na nas (jest naszym dzieckiem), wywołuje `clone()` ->

`do_fork()` -> `copy_process()`

`exec()` ładuje kod i zaczyna wykonywanie

`spawn` = `fork` + `exec`

Bardziej szczegółowy opis:

1. `fork()` – tworzy proces dziecko będący kopią procesu wywołującego tę funkcję (różnica w PID, PPID, niektórych zasobach oraz statystykach, które nie są dziedziczone), implementowany poprzez funkcję systemową `clone()`, która wywołuje funkcję `do_fork()` odpowiedzialną za większość prac związanych z tworzeniem nowego procesu, `do_fork()` wywołuje m.in. funkcję `copy_process()`. Gdy `copy_process()` zakończyła się bezbłędnie – nowy proces jest budzony i uruchamiany.

2. `exec()` – ładuje kod wykonywalny do przestrzeni adresowej i rozpoczyna jego działanie

COW - Technika odwlekająca bądź zapobiegająca kopiowaniu danych (każdej strony pamięci w przestrzeni

adresowej danego procesu) w momencie tworzenia procesu.

3. przebieg `fork()`:

- `dup_task_struct()` – tworzy nowy stos kernela, `thread_info`, `task_struct` (deskryptor procesu dziecka jest identyczny z deskryptorem procesu rodzica),

- sprawdza czy nowy proces nie wyczerpie zasobów przeznaczonych dla bieżącego użytkownika,

- ustawia niektóre informacje w deskrytorze procesu dziecka na wartości domyślne, np. statystyki.

Większość informacji zawartych w `task_struct` pozostaje jednak niezmieniona,

- stan nowego procesu ustawiany jest na `TASK_UNINTERRUPTIBLE` – zapewnia, że nie jest on jeszcze uruchomiony,

- `copy_flags()` – aktualizacja pola `flags` dla procesu dziecka, `•+ PF_SUPERPRIV = 0, PF_FORNOEXEC=1`,

- `alloc_pid()` – przypisanie nowego PID,

- W zależności od przekazanych flag do funkcji `clone()` – duplikowane bądź współdzielone używane zasoby, informacje dotyczące systemu plików, przestrzeni adresowej, przestrzeni nazw oraz signal handlers,

- zwraca wskaźnik do nowopowstałego procesu dziecka

50. Opisz CFS.

CFS - scheduler procesów: **Completely Fair Scheduler**

- oblicza jak długo proces powinien pracować na podstawie wszystkich procesów

- bierze pod uwagę priorytet

- każdy proces działa przez swoją wagę / sumę wszystkich wag
- ustalone targeted latency - czas który dzielimy na wszystkie procesy (mały - lepsza interaktywność ale szamotanie)
- liczy się tylko różnica między priorytetami procesów

51. Zaprezentuj schematycznie i opisz budowę kernela w systemie Linux.

- nie posiada dostępu do biblioteki C ani do standardowych nagłówków języka C, zaimplementowany w GNU C (gcc, Intel C),
- Jądro monolityczne+moduły
- brak ochrony pamięci, (to chyba nie jest do końca prawda)
- posiada trudności z wykonywaniem operacji zmiennoprzecinkowych,
- dla każdego procesu przeznacza taki sam relatywnie mały stos (x86 – konfigurowalny podczas kompilacji – 4KB lub 8KB – historycznie)
- stos kernela zawiera 2 strony - 8KB na 32 bitowych – 16KB na 64 bitowych architekturach),
- podatny na wyścigi - synchronizacja oraz współbieżność (musi obsługiwać asynchronicznie nadchodzące przerwania, możliwe jest wyłączenie procesu, wspiera SMP),
- powinien zapewniać przenośność na różne architektury,
- ostatnia stabilna wersja Linux kernel 2.6.37 (19.01.11)

wady user-space: ([tu odpowiedź na pytanie](#))

- Niedostępność przerwań (dla niektórych platform owszem – np. vm86 dla architektury i386)
- Bezpośredni dostęp do pamięci jest możliwy tylko poprzez mmaping (/dev/mem) co może wykonać tylko uprzywilejowany użytkownik.
- Dostęp do portów I/O jest możliwy tylko po wywołaniu ioperm lub iopl. Nie wszystkie platformy wspierają takie wołanie systemowe, a dostęp do /dev/port może być zbyt powolny.
- Wołania systemowe oraz pliki urządzeń dostępne są tylko dla uprzywilejowanych użytkowników
- Czas reakcji jest wydłużony z uwagi na konieczność przełączenia kontekstu przy przesyłaniu danych bądź poleceniu wykonania akcji pomiędzy klientem i urządzeniem.
- Jeżeli urządzenie zostało zrzuczone (swapped) na dysk, czas reakcji jest bardzo długi. Pomóc może użycie wołania systemowego mlock lecz z reguły należy zablokować wiele stron pamięci (programy działające w user-space zależne są od sporego kodu zawartego w bibliotekach).
- Operacja mlock także jest zarezerwowana dla użytkowników uprzywilejowanych.
- Większość najważniejszych urządzeń nie może być obsługiwanych z poziomu użytkownika (włączając w to interfejsy sieciowe oraz urządzenia blokowe)

53. Slab allocator i jego możliwe zachowania.

- Slab allocator dzieli różne obiekty na grupy zwane cache'ami. Każdy z tych cache'ów przechowuje obiekty określonego typu (np. oddzielny cache jest dla task_struct i oddzielny cache dla inode).
- Cache te są podzielone w serie (ang. slabs)
- Slab składa się z pojedynczej strony pamięci.
- Każdy slab zawiera pewną ilość obiektów, które są zcache'owanymi strukturami danych.
- Stany slab to: pełen (wszystkie obiekty są zaalokowane), połowicznie zajęty bądź wolny (wszystkie obiekty są wolne).
- Gdy kernel żąda nowego obiektu: połowicznie zajęty, wolny. Gdy nie istnieje wolny slab, tworzony jest nowy.

Zachowania:

- SLAB_HWCACHE_ALIGN – bezpośrednio przypisanie każdego obiektu w slabie do określonego pola cache – zabezpiecza przed wskazywaniem większej ilości obiektów na tę samą pozycję w cache
- SLAB_POISON – wypełnienie slaba znaną wartością (a5a5a5a5) w celu wyłapania próby dostępu do niezainicjowanej pamięci (poisoning)
- SLAB_RED_ZONE – wstawienie stref dookoła zaalokowanej pamięci w celu wykrycia przepełnienia buforu.
- SLAB_PANIC – w przypadku, gdy alokacja zawiedzie slab allocator panikuje☹. Używane, gdy alokacja musi zakończyć się sukcesem (np. podczas alokacji struktury VMA w momencie startu systemu).

- SLAB_CACHE_DMA – nowego slaba należy zaalokować w pamięci, która może być dostępna przy pomocy trybu DMA (czyli gdy nowo alokowany obiekt wykorzystywany jest do DMA i musi znajdować się w strefie ZONE_DMA).

54. Przedstaw szczegółowo oraz porównaj metody alokacji pamięci na poziomie kernela w systemie Linux.

- niskopoziomowe oparte na struct page * alloc_pages(gfp_t gfp_mask, unsigned int order)
- kmalloc() - wyższy poziom, gdy nie zależy nam na pobraniu konkretnej strony (działanie podobne do malloc() po stronie użytkownika), gwarantuje zwrot stron fizycznie ciągłych
- vmalloc() - jak kmalloc(), ale zaalokowana pamięć jest wirtualnie ciągła (niekoniecznie fizycznie)
- slab allocator - cache'owanie często używanych struktur
- Gdy potrzebujemy ciągłych fizycznie stron – wykorzystujemy alokatory niskopoziomowe bądź kmalloc()
- Gdy chcemy alokować na dalekich obszarach pamięci alloc_pages() – kmap() w celu odwzorowania dalekiej pamięci do przestrzeni adresowej kernela
- Gdy niepotrzebna fizycznie ciągła pamięć – tylko wirtualnie – wykorzystujemy vmalloc()
- Gdy tworzymy i usuwamy wiele dużych struktur – należy rozważyć slab cache.
- Slab allocator obsługuje także obiekty cacheowane dla każdego procesora osobno.

55. Przedstaw działanie wybranych flag GFP.

- Modyfikatory akcji:
 - __GFP_WAIT – allocator może przejść w stan uśpienia
 - __GFP_HIGH – allocator może korzystać z awaryjnych pól
 - __GFP_REPEAT – allocator powtarza próbę, jeżeli się nie powiodła
 - __GFP_COLD – allocator powinien wykorzystać zimne strony cache (czyli strony na dysku - ciepłe to te w pamięci)
- Modyfikatory strefy (domyślnie alokowane w strefie ZONE_NORMAL):
 - __GFP_DMA – alokuje tylko w strefie ZONE_DMA
 - __GFP_DMA32 – alokuje tylko w strefie ZONE_DMA32
 - __GFP_HIGHMEM – alokuje w strefie ZONE_HIGHMEM bądź w ZONE_NORMAL
- Typy:
 - GFP_ATOMIC – wysoki priorytet, nie może przejść w stan uśpienia. Wykorzystywana np. przy obsłudze przerwań.
 - GFP_NOWAIT – jak w GFP_ATOMIC, tylko nie może wykorzystywać pamięci z zakresu highmem
 - GFP_NOIO – ale nie może inicjować I/O do dysku. Wykorzystywane przy obsłudze dysku, gdy nie można generować więcej żądań do dysku.
 - GFP_NOFS – może inicjować I/O do dysku, ale nie może inicjować operacji na

systemie pliku. Wykorzystywane w kodzie obsługi systemu plików, gdy nie można rozpocząć kolejnej operacji na systemie plików.

- GFP_KERNEL – standardowa alokacja. Wykorzystywana w kodzie procesu, gdy bezpiecznie jest uśpić daną operację. (domyślny wybór)
- GFP_USER – alokacja dla procesów użytkownika
- GFP_HIGHUSER – jak w przypadku GFP_USER
- GFP_DMA – alokacja dla strefy ZONE_DMA, gdy urządzenia wymagają pamięci zdolnej do pracy w trybie DMA

56. Co to jest etyka? :)

dział filozofii, zajmujący się badaniem moralności i tworzeniem systemów myślowych, z których można wyprowadzać zasady moralne.